

SUPPLY MANAGEMENT SYSTEM — Functional Specification Document

FSD Addendum 38 — Point of Sale Module

Property	Value
Document ID	SMS-FSD-ADD-038
Version	1.0
Date	2026-10-10
Status	Draft
Author	System Architect
Classification	Internal — Development
Depends On	SMS-FSD-ADD-037 (Module Registry, Enablement Layer, Customer Master, Sync Foundation)
Architecture	.NET 8 / EF Core 8 / SQL Server 2022 / React 18 PWA / Dexie.js / SignalR / Hangfire
Change Items	12 changes (C1-C12)
Impact	20 migrations, 14 new tables, 3 modified tables, 1 new schema, 1 SignalR hub, 1 PWA client, 1 sync engine

Table of Contents

- 1. Purpose & Scope
- 2. C1 — POS Schema & Store Hierarchy
- 3. C2 — Terminal & Device Registration
- 4. C3 — Shift Lifecycle & Cash Management
- 5. C4 — Cashier Sessions & Shared-PIN Model
- 6. C5 — POS Orders & Line Items
- 7. C6 — Payment Recording & Hybrid Payment Model
- 8. C7 — Product POS Extension & Catalog Sync
- 9. C8 — Offline Credit Threshold & Customer Sync

10. C9 — Inventory Integration (Stock Deduction)
 11. C10 — Finance Integration (Journal Entries)
 12. C11 — SignalR Real-Time Hub
 13. C12 — PWA Client Architecture (React + Dexie + Service Worker)
 14. Database Migrations
 15. API Endpoints
 16. Permission Claims
 17. POS Screen Design & UI Specifications
 18. Business Rules
 19. Test Scenarios
 20. Development Phases
 21. Future Enhancements
-

1. Purpose & Scope

This addendum introduces the Point of Sale (POS) module — a PWA-based retail selling interface that enables brick-and-mortar stores to process sales transactions, manage shifts and cashier accountability, and synchronize data bidirectionally with the SMS backend. The POS module is the first offline-capable vertical module in SMS, establishing patterns for PWA architecture, sync engines, and real-time communication that future modules (mobile warehouse, field service) will reuse.

1.1 Design Principles

- **Offline-First, Sync-Later** — The POS PWA caches product catalog, customers, and pricing locally via IndexedDB (Dexie.js). Transactions are created offline and synced when connectivity resumes. No sale is ever lost to a network outage.
- **Store → Terminal → Shift → Cashier Session** — A clear hierarchy from physical location down to individual accountability. Each store links 1:1 with an `inventory.Warehouses` record for stock deduction. Shifts track cash in/out for the entire terminal's operational period. Cashier sessions provide per-person accountability within a shift.
- **Shared-PIN Cashier Model** — Terminals are shared devices. Cashiers identify themselves with a 4-6 digit PIN, creating a virtual drawer per cashier. This follows Lightspeed's approach — maximum flexibility for both single-cashier shops and multi-cashier stores.
- **Payment-Recorded with Hybrid Data Model** — Version 1 records the payment method and amount (cash, card, mobile) without PCI scope. The data model includes `provider_type` and `provider_config` columns for future Stripe Terminal / payment gateway integration.

- **Threshold-Based Offline Credit** — Walk-in customers: cash only offline. Registered customers: their credit limit is cached locally with a configurable tolerance. Over-limit transactions are flagged on sync for supervisor review.
- **1:1 Store-Warehouse Mapping** — Each store links to exactly one warehouse (Odoo's approach). Stock deduction happens against that warehouse. Bin management is optionally available if the WAREHOUSE module is enabled.
- **Module Registry Integration** — POS is activated through the Module Registry (Addendum 37). Enabling POS validates dependencies on INVENTORY + FINANCE + CUSTOMERS, sets `is_available = 1` in the catalog, creates the `pos` schema objects, and injects POS navigation into the sidebar.

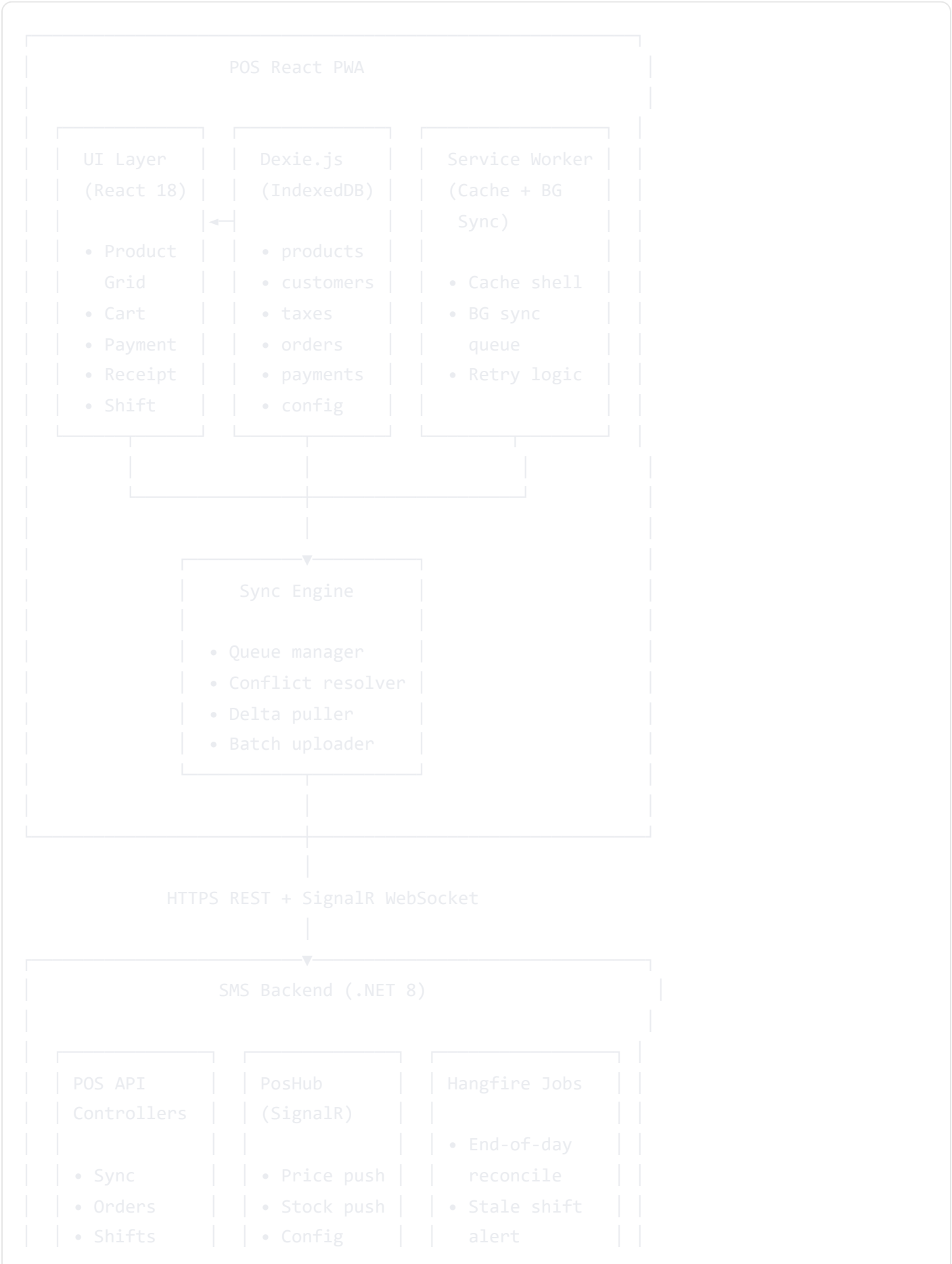
1.2 Target Scope

Included	Excluded (Future Addendum)
Single-store and multi-store retail chain	Restaurant / F&B (table management, kitchen display)
Cashier selling interface (product grid, barcode, cart)	Franchise model (inter-company POS)
Cash, card, mobile payment recording	Integrated payment processing (Stripe, Adyen)
Shift open/close/reconcile	E-Commerce channel integration
Offline transaction support	Loyalty program point redemption
Customer assignment and credit sale	Advanced promotions engine (BOGO, combo deals)
Receipt generation (digital)	Hardware printing (receipt printers, cash drawer kick)
Basic discount (line % and order %)	Multi-currency POS (future — uses org default currency)

1.3 Changes Summary

Change ID	Title	Category
C1	POS Schema & Store Hierarchy	New schema <code>pos</code> , new tables — <code>pos.Stores</code>
C2	Terminal & Device Registration	New table — <code>pos.Terminals</code>
C3	Shift Lifecycle & Cash Management	New tables — <code>pos.Shifts</code> , <code>pos.CashMovements</code>
C4	Cashier Sessions & Shared-PIN Model	New table — <code>pos.CashierSessions</code> , modify <code>auth.Users</code>
C5	POS Orders & Line Items	New tables — <code>pos.PosOrders</code> , <code>pos.PosOrderLines</code>
C6	Payment Recording & Hybrid Payment Model	New table — <code>pos.PosPayments</code>
C7	Product POS Extension & Catalog Sync	New table — <code>pos.ProductPosSettings</code> , modify sync API
C8	Offline Credit Threshold & Customer Sync	New table — <code>pos.CustomerPosSettings</code>
C9	Inventory Integration (Stock Deduction)	Stock movement auto-creation on order completion
C10	Finance Integration (Journal Entries)	Journal entry auto-creation on order completion
C11	SignalR Real-Time Hub	New hub — <code>PosHub</code> for price/stock/config push
C12	PWA Client Architecture	React PWA + Dexie.js + Service Worker + Sync Engine

1.4 Architecture Overview



1.5 Store Hierarchy

```
tenant.Organizations (tenant / brand)
├── pos.Stores (physical locations)
│   ├── 1:1 → inventory.Warehouses (stock source)
│   ├── pos.Terminals (registers / devices)
│   │   └── pos.Shifts (operational periods per terminal)
│   │       ├── pos.CashierSessions (cashier on terminal during shift)
│   │       │   └── pos.PosOrders (sales transactions)
│   │       │       ├── pos.PosOrderLines (line items)
│   │       │       └── pos.PosPayments (payment tenders)
│   │       └── pos.CashMovements (cash in / cash out during shift)
│   └── auth.Users (store staff – cashiers, supervisors)
```

2. C1 — POS Schema & Store Hierarchy

2.1 Rationale

Stores are the top-level POS entity. A brand (organization) may have one or many stores. Each store is a physical retail location linked to exactly one warehouse for inventory operations. Store-level configuration (receipt header, tax settings, operating hours) lives here.

2.2 New Schema

```
sql

CREATE SCHEMA pos;
```

2.3 Entity

2.3.1 **pos.Stores**

Column	Type	Constraints	Description
store_id	BIGINT	PK, IDENTITY	Auto-increment PK
org_id	BIGINT	NOT NULL, FK → tenant.Organizations	Tenant
code	VARCHAR(20)	NOT NULL	Store code: STR- 001, STR-MAIN
name	NVARCHAR(200)	NOT NULL	Store display name: "Downtown Branch"
warehouse_id	BIGINT	NOT NULL, FK → inventory.Warehouses	1:1 linked warehouse for stock operations
address_line1	NVARCHAR(200)	NULL	Store physical address
address_line2	NVARCHAR(200)	NULL	
city	NVARCHAR(100)	NULL	
state_province	NVARCHAR(100)	NULL	
postal_code	VARCHAR(20)	NULL	
country_code	VARCHAR(3)	NULL	ISO 3166-1 alpha-3
phone	VARCHAR(30)	NULL	Store contact number
email	NVARCHAR(200)	NULL	Store contact email
timezone	VARCHAR(50)	NOT NULL, DEFAULT 'UTC'	IANA timezone for shift scheduling

Column	Type	Constraints	Description
currency_code	VARCHAR(3)	NOT NULL, DEFAULT 'USD'	Store operating currency
tax_inclusive_pricing	BIT	NOT NULL, DEFAULT 0	0 = tax exclusive (add on top), 1 = tax inclusive (included in price)
default_tax_rate_id	BIGINT	NULL, FK → lookups.TaxRates	Default tax rate for this store
receipt_header	NVARCHAR(500)	NULL	Custom receipt header text (store name, address, tax reg)
receipt_footer	NVARCHAR(500)	NULL	Custom receipt footer text (return policy, thank you message)
offline_credit_tolerance_pct	DECIMAL(5,2)	NOT NULL, DEFAULT 10.00	Offline credit tolerance percentage above cached limit
auto_close_shift_hours	INT	NOT NULL, DEFAULT 24	Auto-flag shift as stale after N hours
is_active	BIT	NOT NULL, DEFAULT 1	
created_at	DATETIME2(2)	NOT NULL, DEFAULT SYSUTCDATETIME()	
created_by	BIGINT	NOT NULL, FK → auth.Users	
modified_at	DATETIME2(2)	NOT NULL, DEFAULT SYSUTCDATETIME()	
modified_by	BIGINT	NULL, FK → auth.Users	

Column	Type	Constraints	Description
<code>row_version</code>	ROWVERSION	NOT NULL	Optimistic concurrency

Indexes:

- PK: `store_id`
- UQ: `(org_id, code)` — unique store code per tenant
- IX: `(org_id, is_active)` — active store filter
- IX: `(org_id, warehouse_id)` — warehouse lookup
- IX: `(org_id, modified_at)` — sync

EF Core Query Filter:

```
csharp
builder.HasQueryFilter(s => s.OrgId == _currentTenant.OrgId);
```

Business Rule: One warehouse can be linked to at most one store. Enforced at application level (not DB unique constraint, because a warehouse may not have a store). Validation: `if (await _db.Stores.AnyAsync(s => s.WarehouseId == dto.WarehouseId && s.StoreId != storeId)) throw ConflictException`.

3. C2 — Terminal & Device Registration

3.1 Rationale

A terminal is a logical register within a store. It may correspond to a physical device (tablet, PC) or be a logical session point. Terminals have names for identification and can be activated/deactivated. Each terminal can have one active shift at a time.

3.2 Entity

3.2.1 `pos.Terminals`

Column	Type	Constraints	Description
<code>terminal_id</code>	BIGINT	PK, IDENTITY	Auto-increment PK
<code>org_id</code>	BIGINT	NOT NULL, FK → tenant.Organizations	Tenant
<code>store_id</code>	BIGINT	NOT NULL, FK → pos.Stores	Parent store
<code>code</code>	VARCHAR(20)	NOT NULL	Terminal code: <code>T-01</code> , <code>REG-A</code>
<code>name</code>	NVARCHAR(100)	NOT NULL	Display name: "Register 1", "Checkout Counter A"
<code>device_uuid</code>	VARCHAR(100)	NULL	Bound device UUID (set on first PWA activation)
<code>last_seen_at</code>	DATETIME2(2)	NULL	Last heartbeat from PWA client
<code>last_sync_at</code>	DATETIME2(2)	NULL	Last successful data sync
<code>is_active</code>	BIT	NOT NULL, DEFAULT 1	
<code>created_at</code>	DATETIME2(2)	NOT NULL, DEFAULT SYSUTCDATETIME()	
<code>created_by</code>	BIGINT	NOT NULL, FK → auth.Users	
<code>modified_at</code>	DATETIME2(2)	NOT NULL, DEFAULT SYSUTCDATETIME()	
<code>modified_by</code>	BIGINT	NULL, FK → auth.Users	
<code>row_version</code>	ROWVERSION	NOT NULL	Optimistic concurrency

Indexes:

- PK: `terminal_id`
- UQ: `(org_id, store_id, code)` — unique terminal code per store
- IX: `(store_id, is_active)` — active terminals in store
- IX: `(device_uuid)` WHERE device_uuid IS NOT NULL — device lookup

Device Binding Flow:

1. Admin creates terminal in back-office (code, name, store assignment)
 2. Terminal exists with `device_uuid = NULL` (unbound)
 3. Cashier opens PWA URL on the physical device → PWA generates UUID (`crypto.randomUUID()`)
 4. PWA hits `POST /api/pos/terminals/{id}/bind` with the UUID
 5. Backend sets `device_uuid` and returns terminal configuration
 6. Subsequent PWA loads from this device auto-identify via stored UUID
-

4. C3 — Shift Lifecycle & Cash Management

4.1 Rationale

A shift is an operational period on a terminal. It tracks all cash movement from opening to closing, providing accountability for cash differences. The shift lifecycle follows a state machine: DRAFT → OPEN → CLOSING → CLOSED → RECONCILED (or DISPUTED).

4.2 Entities

4.2.1 `pos.Shifts`

Column	Type	Constraints	Description
shift_id	BIGINT	PK, IDENTITY	Auto-increment PK
org_id	BIGINT	NOT NULL, FK → tenant.Organizations	Tenant
store_id	BIGINT	NOT NULL, FK → pos.Stores	Store
terminal_id	BIGINT	NOT NULL, FK → pos.Terminals	Terminal this shift runs on
shift_number	VARCHAR(30)	NOT NULL	Auto-generated: SH-{store_code}-{YYYYMMDD}-{seq}
status	TINYINT	NOT NULL, DEFAULT 0	0=Draft, 1=Open, 2=Closing, 3=Closed, 4=Reconciled, 5=Disputed
opened_by	BIGINT	NOT NULL, FK → auth.Users	Supervisor or cashier who opened
opened_at	DATETIME2(2)	NULL	Actual open time
closed_by	BIGINT	NULL, FK → auth.Users	Who initiated closing
closed_at	DATETIME2(2)	NULL	Actual close time
reconciled_by	BIGINT	NULL, FK → auth.Users	Supervisor who reconciled
reconciled_at	DATETIME2(2)	NULL	Reconciliation time
opening_float	DECIMAL(18,4)	NOT NULL, DEFAULT 0	Cash placed in drawer at shift start
total_cash_sales	DECIMAL(18,4)	NOT NULL, DEFAULT 0	Running total of cash payments received

Column	Type	Constraints	Description
<code>total_card_sales</code>	DECIMAL(18,4)	NOT NULL, DEFAULT 0	Running total of card payments
<code>total_mobile_sales</code>	DECIMAL(18,4)	NOT NULL, DEFAULT 0	Running total of mobile/digital payments
<code>total_credit_sales</code>	DECIMAL(18,4)	NOT NULL, DEFAULT 0	Running total of credit/on-account sales
<code>total_cash_refunds</code>	DECIMAL(18,4)	NOT NULL, DEFAULT 0	Running total of cash refunds
<code>total_card_refunds</code>	DECIMAL(18,4)	NOT NULL, DEFAULT 0	Running total of card refunds
<code>total_cash_in</code>	DECIMAL(18,4)	NOT NULL, DEFAULT 0	Sum of cash-in movements (float top-ups, etc.)
<code>total_cash_out</code>	DECIMAL(18,4)	NOT NULL, DEFAULT 0	Sum of cash-out movements (safe drops, petty cash, etc.)
<code>expected_cash_closing</code>	DECIMAL(18,4)	NOT NULL, DEFAULT 0	Computed: <code>opening_float + total_cash_sales + total_cash_in - total_cash_out - total_cash_refunds</code>
<code>actual_cash_closing</code>	DECIMAL(18,4)	NULL	Physically counted cash at close
<code>cash_variance</code>	DECIMAL(18,4)	NULL	<code>actual_cash_closing - expected_cash_closing</code>
<code>variance_threshold</code>	DECIMAL(18,4)	NOT NULL, DEFAULT 5.00	Maximum acceptable variance before flagging dispute
<code>variance_note</code>	NVARCHAR(500)	NULL	Explanation for variance
<code>order_count</code>	INT	NOT NULL, DEFAULT 0	Total orders processed

Column	Type	Constraints	Description
<code>refund_count</code>	INT	NOT NULL, DEFAULT 0	Total refunds processed
<code>gross_total</code>	DECIMAL(18,4)	NOT NULL, DEFAULT 0	Sum of all order totals before refunds
<code>net_total</code>	DECIMAL(18,4)	NOT NULL, DEFAULT 0	Gross minus refunds
<code>is_synced</code>	BIT	NOT NULL, DEFAULT 0	Whether this shift has been fully synced to backend
<code>created_at</code>	DATETIME2(2)	NOT NULL, DEFAULT SYSUTCDATETIME()	
<code>modified_at</code>	DATETIME2(2)	NOT NULL, DEFAULT SYSUTCDATETIME()	
<code>row_version</code>	ROWVERSION	NOT NULL	

Indexes:

- PK: `shift_id`
- UQ: `(org_id, shift_number)` — unique shift number per tenant
- IX: `(terminal_id, status)` — find active shift on terminal
- IX: `(store_id, status)` — store-level shift overview
- IX: `(org_id, opened_at)` — date range queries

Shift State Machine:

Expected Cash Formula:

```
expected_cash_closing = opening_float
                        + total_cash_sales
                        + total_cash_in
                        - total_cash_out
                        - total_cash_refunds
```

4.2.2 pos.CashMovements

Non-sale cash movements during a shift (safe drops, float top-ups, petty cash withdrawals).

Column	Type	Constraints	Description
cash_movement_id	BIGINT	PK, IDENTITY	Auto-increment PK
org_id	BIGINT	NOT NULL, FK → tenant.Organizations	Tenant
shift_id	BIGINT	NOT NULL, FK → pos.Shifts	Parent shift
direction	TINYINT	NOT NULL	0 = CashIn, 1 = CashOut
amount	DECIMAL(18,4)	NOT NULL	Always positive
reason	TINYINT	NOT NULL	0=FloatTopUp, 1=SafeDrop, 2=PettyCash, 3=ChangeRun, 4=Other
note	NVARCHAR(300)	NULL	Free-text explanation
performed_by	BIGINT	NOT NULL, FK → auth.Users	Who performed the movement
performed_at	DATETIME2(2)	NOT NULL, DEFAULT SYSUTCDATETIME()	
local_id	VARCHAR(50)	NULL	PWA-generated UUID for offline dedup
is_synced	BIT	NOT NULL, DEFAULT 0	
created_at	DATETIME2(2)	NOT NULL, DEFAULT SYSUTCDATETIME()	

Indexes:

- PK: `cash_movement_id`
- IX: `(shift_id, direction)` — list movements per shift
- UQ: `(org_id, local_id)` WHERE local_id IS NOT NULL — offline dedup

5. C4 — Cashier Sessions & Shared-PIN Model

5.1 Rationale

Multiple cashiers can operate on the same terminal during a shift using a 4–6 digit PIN. Each PIN entry creates or resumes a cashier session, which provides a virtual drawer per cashier for accountability. This follows Lightspeed's shared-PIN model — the most flexible approach for both single-cashier shops and multi-cashier retail chains.

5.2 User Table Modification

Add POS-specific columns to `auth.Users`:

Column	Type	Constraints	Description
<code>pos_pin</code>	VARCHAR(100)	NULL	Hashed 4–6 digit PIN (bcrypt). NULL = user has no POS access via PIN
<code>pos_role</code>	TINYINT	NULL	0=Cashier, 1=Supervisor. NULL = not a POS user
<code>default_store_id</code>	BIGINT	NULL, FK → pos.Stores	Default store assignment

PIN Security: The PIN is hashed with bcrypt (cost factor 10). The PIN is never stored or transmitted in plaintext. PIN entry at the terminal sends the plaintext PIN over HTTPS to the backend for verification, or in offline mode, compares against a locally cached hash.

PIN Uniqueness: PINs must be unique within a store (not globally). Validation: no two active users assigned to the same store may share the same PIN.

5.3 Entity

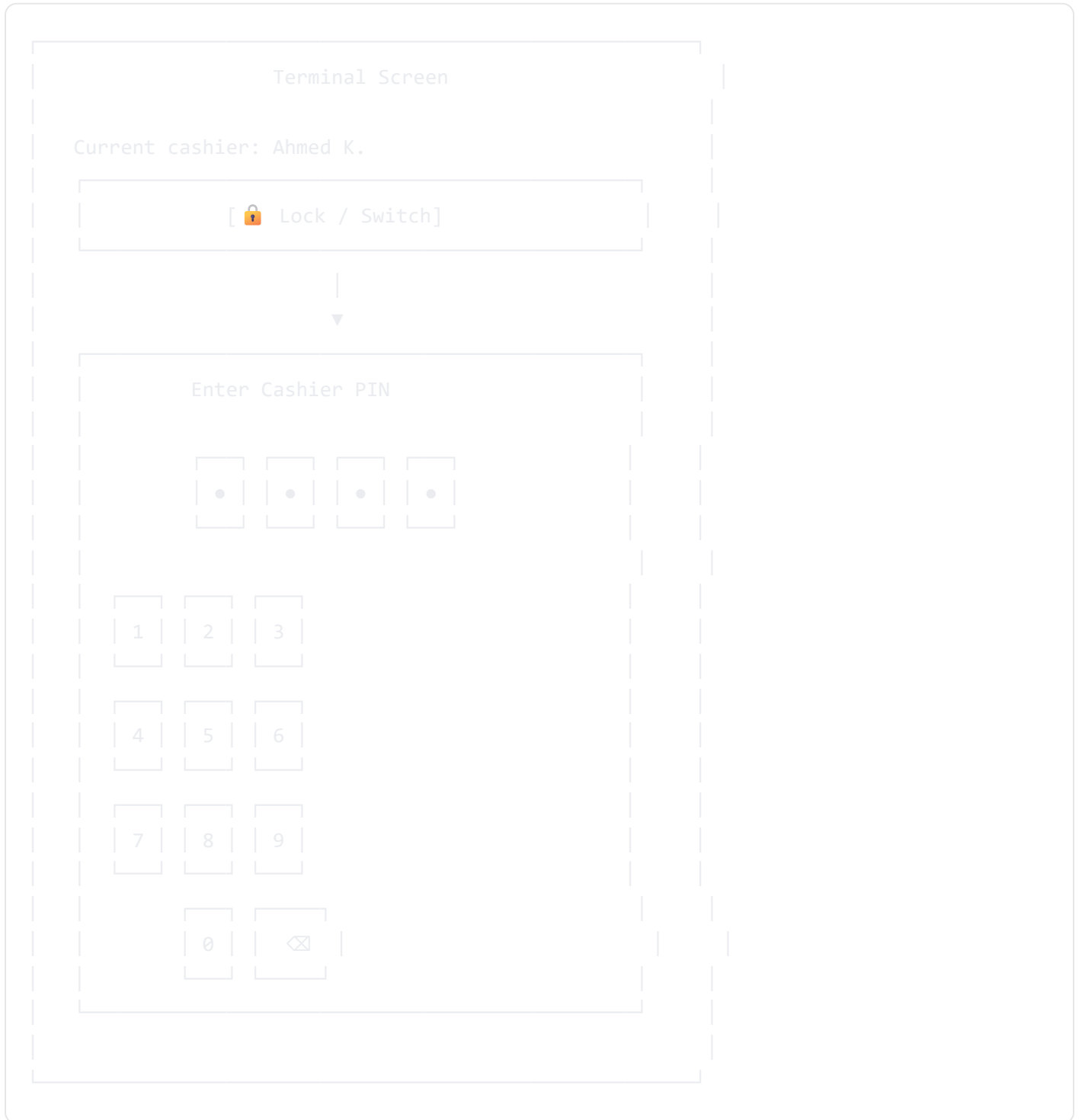
5.3.1 `pos.CashierSessions`

Column	Type	Constraints	Description
<code>session_id</code>	BIGINT	PK, IDENTITY	Auto-increment PK
<code>org_id</code>	BIGINT	NOT NULL, FK → tenant.Organizations	Tenant
<code>shift_id</code>	BIGINT	NOT NULL, FK → pos.Shifts	Parent shift
<code>terminal_id</code>	BIGINT	NOT NULL, FK → pos.Terminals	Terminal (denormalized for query perf)
<code>user_id</code>	BIGINT	NOT NULL, FK → auth.Users	Cashier user
<code>started_at</code>	DATETIME2(2)	NOT NULL, DEFAULT SYSUTCDATETIME()	When cashier signed in
<code>ended_at</code>	DATETIME2(2)	NULL	When cashier signed out or was replaced
<code>cash_sales</code>	DECIMAL(18,4)	NOT NULL, DEFAULT 0	Cash collected by this cashier
<code>card_sales</code>	DECIMAL(18,4)	NOT NULL, DEFAULT 0	Card payments by this cashier
<code>mobile_sales</code>	DECIMAL(18,4)	NOT NULL, DEFAULT 0	Mobile payments by this cashier
<code>credit_sales</code>	DECIMAL(18,4)	NOT NULL, DEFAULT 0	Credit sales by this cashier
<code>refund_total</code>	DECIMAL(18,4)	NOT NULL, DEFAULT 0	Refunds by this cashier
<code>order_count</code>	INT	NOT NULL, DEFAULT 0	Orders processed by this cashier
<code>is_active</code>	BIT	NOT NULL, DEFAULT 1	Currently active session
<code>local_id</code>	VARCHAR(50)	NULL	Offline dedup ID
<code>created_at</code>	DATETIME2(2)	NOT NULL, DEFAULT SYSUTCDATETIME()	

Indexes:

- PK: `session_id`
- IX: `(shift_id, is_active)` — find active session on shift
- IX: `(user_id, started_at)` — cashier session history
- UQ: `(org_id, local_id)` WHERE local_id IS NOT NULL — offline dedup

Cashier Switch Flow:



Actions on PIN entry:

1. Previous cashier's session → `ended_at = now`, `is_active = 0`
 2. New cashier session created for the entered PIN's user
 3. Cart is cleared (no cross-cashier cart carry-over)
 4. If same PIN re-entered → session resumes (no new session created)
-

6. C5 — POS Orders & Line Items

6.1 Rationale

POS orders are the core transaction entity. Unlike traditional sales orders, POS orders are created and completed in a single interaction (immediate fulfillment). They originate offline on the PWA and sync to the backend.

6.2 Entities

6.2.1 `pos.PosOrders`

Column	Type	Constraints	Description
<code>order_id</code>	BIGINT	PK, IDENTITY	Auto-increment PK
<code>org_id</code>	BIGINT	NOT NULL, FK → tenant.Organizations	Tenant
<code>store_id</code>	BIGINT	NOT NULL, FK → pos.Stores	Store
<code>terminal_id</code>	BIGINT	NOT NULL, FK → pos.Terminals	Terminal
<code>shift_id</code>	BIGINT	NOT NULL, FK → pos.Shifts	Shift
<code>session_id</code>	BIGINT	NOT NULL, FK → pos.CashierSessions	Cashier session
<code>order_number</code>	VARCHAR(30)	NOT NULL	Auto-generated: <code>POS-{store_code}-{YYYYMMDD}-{seq}</code>
<code>order_type</code>	TINYINT	NOT NULL, DEFAULT 0	0=Sale, 1=Return, 2=Exchange
<code>status</code>	TINYINT	NOT NULL, DEFAULT 0	0=Draft, 1=Completed, 2=Voided, 3=Returned, 4=PartialReturn
<code>customer_id</code>	BIGINT	NULL, FK → customers.Customers	NULL = walk-in (uses default Walk-In Customer)
<code>customer_name</code>	NVARCHAR(200)	NULL	Snapshot of customer name at order time
<code>line_count</code>	INT	NOT NULL, DEFAULT 0	Number of line items
<code>subtotal</code>	DECIMAL(18,4)	NOT NULL, DEFAULT 0	Sum of line totals before order-level discount
<code>discount_type</code>	TINYINT	NOT NULL, DEFAULT 0	0=None, 1=Percentage, 2=FixedAmount

Column	Type	Constraints	Description
<code>discount_value</code>	DECIMAL(18,4)	NOT NULL, DEFAULT 0	Discount percentage or fixed amount
<code>discount_amount</code>	DECIMAL(18,4)	NOT NULL, DEFAULT 0	Computed discount amount
<code>tax_total</code>	DECIMAL(18,4)	NOT NULL, DEFAULT 0	Sum of line tax amounts
<code>grand_total</code>	DECIMAL(18,4)	NOT NULL, DEFAULT 0	<code>subtotal</code> - <code>discount_amount</code> + <code>tax_total</code>
<code>paid_amount</code>	DECIMAL(18,4)	NOT NULL, DEFAULT 0	Sum of all payments
<code>change_amount</code>	DECIMAL(18,4)	NOT NULL, DEFAULT 0	Cash overpayment returned
<code>balance_due</code>	DECIMAL(18,4)	NOT NULL, DEFAULT 0	<code>grand_total</code> - <code>paid_amount</code> (0 for completed, >0 for credit)
<code>payment_status</code>	TINYINT	NOT NULL, DEFAULT 0	0=Unpaid, 1=Paid, 2=PartialPaid, 3=CreditSale
<code>original_order_id</code>	BIGINT	NULL, FK → self	For returns: the original sale order
<code>return_reason</code>	NVARCHAR(300)	NULL	Reason for return
<code>note</code>	NVARCHAR(500)	NULL	Order-level note
<code>local_id</code>	VARCHAR(50)	NOT NULL	PWA-generated UUID for offline creation + dedup
<code>is_offline</code>	BIT	NOT NULL, DEFAULT 0	Was this order created while terminal was offline?
<code>synced_at</code>	DATETIME2(2)	NULL	When this order was synced to backend
<code>created_at</code>	DATETIME2(2)	NOT NULL, DEFAULT SYSUTCDATETIME()	

Column	Type	Constraints	Description
<code>created_by</code>	BIGINT	NOT NULL, FK → auth.Users	Cashier who created
<code>modified_at</code>	DATETIME2(2)	NOT NULL, DEFAULT SYSUTCDATETIME()	
<code>row_version</code>	ROWVERSION	NOT NULL	

Indexes:

- PK: `order_id`
- UQ: `(org_id, order_number)` — unique order number per tenant
- UQ: `(org_id, local_id)` — offline dedup
- IX: `(store_id, created_at)` — store sales by date
- IX: `(shift_id, status)` — orders within a shift
- IX: `(session_id)` — cashier's orders
- IX: `(customer_id)` WHERE customer_id IS NOT NULL — customer order history
- IX: `(org_id, synced_at)` WHERE synced_at IS NULL — pending sync queue

Order Number Generation (Backend):

```
csharp
// Atomic sequence per store per day
var today = DateOnly.FromDateTime(DateTime.UtcNow);
var seq = await _db.PosOrders
    .Where(o => o.StoreId == storeId && o.CreatedAt.Date == today.ToDateTime(TimeOnly.MinValue))
    .CountAsync() + 1;
return $"POS-{storeCode}-{today:yyyyMMdd}-{seq:D4}";
```

Offline Order Numbers: When offline, the PWA assigns a temporary number `POS-{store}-OFF-{uuid_short}`. On sync, the backend replaces it with the canonical sequential number.

6.2.2 `pos.PosOrderLines`

Column	Type	Constraints	Description
<code>line_id</code>	BIGINT	PK, IDENTITY	Auto-increment PK
<code>org_id</code>	BIGINT	NOT NULL, FK → tenant.Organizations	Tenant
<code>order_id</code>	BIGINT	NOT NULL, FK → pos.PosOrders	Parent order
<code>line_number</code>	INT	NOT NULL	Sequential within order (1, 2, 3...)
<code>product_id</code>	BIGINT	NOT NULL, FK → inventory.Products	Product sold
<code>product_name</code>	NVARCHAR(200)	NOT NULL	Snapshot of product name at sale time
<code>product_sku</code>	VARCHAR(50)	NULL	Snapshot of SKU
<code>barcode</code>	VARCHAR(50)	NULL	Snapshot of barcode scanned
<code>quantity</code>	DECIMAL(18,4)	NOT NULL	Quantity sold (supports fractional for weight items)
<code>uom_id</code>	BIGINT	NOT NULL, FK → lookups.UnitOfMeasures	Unit of measure
<code>unit_price</code>	DECIMAL(18,4)	NOT NULL	Price per unit at time of sale
<code>discount_type</code>	TINYINT	NOT NULL, DEFAULT 0	0=None, 1=Percentage, 2=FixedAmount
<code>discount_value</code>	DECIMAL(18,4)	NOT NULL, DEFAULT 0	Line-level discount
<code>discount_amount</code>	DECIMAL(18,4)	NOT NULL, DEFAULT 0	Computed: <code>unit_price * quantity * (discount_value/100)</code> or fixed
<code>tax_rate_id</code>	BIGINT	NULL, FK → lookups.TaxRates	Applied tax rate

Column	Type	Constraints	Description
<code>tax_percentage</code>	DECIMAL(5,2)	NOT NULL, DEFAULT 0	Snapshot of tax rate at sale time
<code>tax_amount</code>	DECIMAL(18,4)	NOT NULL, DEFAULT 0	Computed tax
<code>line_total</code>	DECIMAL(18,4)	NOT NULL	<code>(unit_price * quantity)</code> <code>- discount_amount +</code> <code>tax_amount</code>
<code>is_return_line</code>	BIT	NOT NULL, DEFAULT 0	True if this line is a return (negative quantity)
<code>original_line_id</code>	BIGINT	NULL, FK → self	For returns: the original line
<code>note</code>	NVARCHAR(300)	NULL	Line note
<code>created_at</code>	DATETIME2(2)	NOT NULL, DEFAULT SYSUTCDATETIME()	

Indexes:

- PK: `line_id`
- IX: `(order_id, line_number)` — line sequence
- IX: `(product_id, created_at)` — product sales history

7. C6 — Payment Recording & Hybrid Payment Model

7.1 Rationale

Version 1 records the payment method and amount without connecting to a payment gateway (no PCI scope). An order can have multiple payment tenders (split payment: part cash, part card). The data model includes columns for future payment provider integration.

7.2 Entity

7.2.1 `pos.PosPayments`

Column	Type	Constraints	Description
payment_id	BIGINT	PK, IDENTITY	Auto-increment PK
org_id	BIGINT	NOT NULL, FK → tenant.Organizations	Tenant
order_id	BIGINT	NOT NULL, FK → pos.PosOrders	Parent order
method	TINYINT	NOT NULL	0=Cash, 1=Card, 2=MobileWallet, 3=BankTransfer, 4=Credit (on-account), 5=Cheque
amount	DECIMAL(18,4)	NOT NULL	Amount tendered
change_given	DECIMAL(18,4)	NOT NULL, DEFAULT 0	Change returned (cash only)
reference	VARCHAR(100)	NULL	Card last-4, transaction ref, wallet ID — manual entry
provider_type	VARCHAR(50)	NULL	Future: stripe_terminal, adyen, square
provider_transaction_id	VARCHAR(200)	NULL	Future: gateway transaction ID
provider_status	TINYINT	NULL	Future: 0=Pending, 1=Authorized, 2=Captured, 3=Failed, 4=Refunded
provider_config	NVARCHAR(MAX)	NULL	Future: JSON config/response from provider
is_refund	BIT	NOT NULL, DEFAULT 0	True if this is a refund payment

Column	Type	Constraints	Description
<code>local_id</code>	VARCHAR(50)	NULL	Offline dedup ID
<code>created_at</code>	DATETIME2(2)	NOT NULL, DEFAULT SYSUTCDATETIME()	

Indexes:

- PK: `payment_id`
- IX: `(order_id)` — payments per order
- IX: `(org_id, method, created_at)` — payment method reporting
- UQ: `(org_id, local_id)` WHERE `local_id` IS NOT NULL — offline dedup

Split Payment Example:

```
Order Grand Total: $150.00
Payment 1: Cash    $100.00 (change_given = $0)
Payment 2: Card    $50.00 (reference = "****4242")
-----
Total Paid:        $150.00
Balance Due:        $0.00
Payment Status:     Paid (1)
```

Credit Sale Example:

```
Order Grand Total: $200.00
Payment 1: Credit  $200.00 (method=4, on-account to customer_id=45)
-----
Total Paid:        $200.00 (booked as receivable)
Balance Due:        $0.00
Payment Status:     CreditSale (3)

→ customers.Customers.current_balance += $200.00
```

8. C7 — Product POS Extension & Catalog Sync

8.1 Rationale

Products need POS-specific attributes (barcode, POS display name, sale price, POS category) that don't belong on the core product table. Following the extension table pattern from

Addendum 37, `pos.ProductPosSettings` provides a 1:1 satellite table.

8.2 Entity

8.2.1 `pos.ProductPosSettings`

Column	Type	Constraints	Description
<code>product_id</code>	BIGINT	PK, FK → inventory.Products	1:1 with product
<code>barcode</code>	VARCHAR(50)	NULL	Product barcode (EAN-13, UPC-A, Code 128, etc.)
<code>pos_name</code>	NVARCHAR(100)	NULL	Short display name for POS grid (if different from product name)
<code>sale_price</code>	DECIMAL(18,4)	NULL	POS selling price (overrides product base price if set)
<code>pos_category_id</code>	BIGINT	NULL, FK → lookups.Categories	POS-specific category for grid layout
<code>color</code>	VARCHAR(7)	NULL	Hex color for POS product tile:  <code>#FF5733</code>
<code>image_url</code>	VARCHAR(500)	NULL	Product image URL for POS grid tile
<code>is_weighable</code>	BIT	NOT NULL, DEFAULT 0	Requires quantity by weight at POS
<code>is_favorite</code>	BIT	NOT NULL, DEFAULT 0	Show on quick-access favorites grid
<code>sort_order</code>	INT	NOT NULL, DEFAULT 0	Sort within POS category
<code>is_pos_active</code>	BIT	NOT NULL, DEFAULT 1	Available on POS (within is_sellable products)
<code>modified_at</code>	DATETIME2(2)	NOT NULL, DEFAULT SYSUTCDATETIME()	For delta sync

Indexes:

- PK: `product_id`
- IX: `(barcode)` WHERE barcode IS NOT NULL — barcode scan lookup
- IX: `(pos_category_id, sort_order)` — category grid layout
- IX: `(is_favorite)` WHERE is_favorite = 1 — favorites filter
- IX: `(modified_at)` — delta sync

Catalog Sync Enhancement:

The existing `GET /api/sync/catalog?since={iso8601}` (Addendum 37 §13.5) is extended to include POS settings:

json

```

{
  "products": [
    {
      "productId": 42,
      "name": "Arabica Coffee Beans 250g",
      "sku": "COF-ARB-250",
      "basePrice": 12.99,
      "isSellable": true,
      "isStockable": true,
      "modifiedAt": "2026-10-10T14:30:00Z",
      "posSettings": {
        "barcode": "5901234567890",
        "posName": "Arabica 250g",
        "salePrice": 14.99,
        "posCategoryId": 15,
        "color": "#6F4E37",
        "isWeighable": false,
        "isFavorite": true,
        "sortOrder": 1,
        "isPosActive": true
      },
      "stockLevel": {
        "warehouseId": 7,
        "available": 45
      }
    }
  ],
  "categories": [ ... ],
  "taxRates": [ ... ],
  "uoms": [ ... ],
  "syncTimestamp": "2026-10-10T14:31:00Z"
}

```

Stock Level in Sync: For POS sync, the response includes `stockLevel` for the store's linked warehouse. This is a read-only snapshot — the PWA displays it but does not enforce stock blocking (negative stock is allowed and flagged on sync).

9. C8 — Offline Credit Threshold & Customer Sync

9.1 Rationale

When offline, the POS needs to make credit-sale decisions without real-time balance queries. Each customer's credit limit and current balance are cached locally. A configurable tolerance per store allows sales slightly over the cached limit, with post-sync reconciliation.

9.2 Entity

9.2.1 `pos.CustomerPosSettings`

Column	Type	Constraints	Description
<code>customer_id</code>	BIGINT	PK, FK → customers.Customers	1:1 with customer
<code>allow_pos_credit</code>	BIT	NOT NULL, DEFAULT 0	Whether this customer can make credit purchases at POS
<code>offline_credit_limit_override</code>	DECIMAL(18,4)	NULL	Per-customer offline credit limit override (NULL = use customer.credit_limit)
<code>max_single_credit_order</code>	DECIMAL(18,4)	NULL	Maximum single credit order amount
<code>requires_id_for_credit</code>	BIT	NOT NULL, DEFAULT 0	Cashier must verify ID for credit sale
<code>notes</code>	NVARCHAR(300)	NULL	POS-specific notes about this customer
<code>modified_at</code>	DATETIME2(2)	NOT NULL, DEFAULT SYSUTCDATETIME()	For delta sync

Indexes:

- PK: `customer_id`
- IX: `(allow_pos_credit)` WHERE allow_pos_credit = 1

9.3 Offline Credit Logic

Decision Tree (runs locally on PWA):

1. Is customer Walk-In (`customer_type = 0`)?
YES → Credit sale blocked. Cash / Card / Mobile only.
2. Is terminal ONLINE?
YES → Query real-time balance from backend.
Check: $\text{current_balance} + \text{order_total} \leq \text{credit_limit}$
If over → Block with message.
3. Terminal is OFFLINE:
 - a. Is `allow_pos_credit = true`?
NO → Credit sale blocked.
 - b. Get cached values:
`cached_limit = offline_credit_limit_override ?? customer.credit_limit`
`cached_balance = last-synced customer.current_balance`
`tolerance = store.offline_credit_tolerance_pct`
`effective_limit = cached_limit × (1 + tolerance / 100)`
 - c. Check: $\text{cached_balance} + \text{order_total} \leq \text{effective_limit}$?
YES → Allow credit sale. Flag as `offline_credit = true`.
NO → Block with message showing cached limit.
4. On SYNC:
Backend recalculates actual balance.
If `actual_balance > credit_limit`:
 - Flag order as `CREDIT_OVER_LIMIT`
 - Create supervisor notification
 - Do NOT void the order (goods already given)

9.4 Customer Sync Enhancement

The existing `GET /api/sync/customers?since={iso8601}` (Addendum 37 §13.5) is extended to include POS settings:

json

```
{
  "customers": [
    {
      "customerId": 45,
      "code": "C-00045",
      "name": "Sarah Mitchell",
      "customerType": 1,
      "phone": "+1-555-0123",
      "creditLimit": 5000.00,
      "currentBalance": 1200.00,
      "modifiedAt": "2026-10-09T18:00:00Z",
      "posSettings": {
        "allowPosCredit": true,
        "offlineCreditLimitOverride": null,
        "maxSingleCreditOrder": 1000.00,
        "requiresIdForCredit": false
      }
    }
  ],
  "syncTimestamp": "2026-10-10T14:31:00Z"
}
```

10. C9 — Inventory Integration (Stock Deduction)

10.1 Rationale

When a POS order is completed (status = Completed), stock must be deducted from the store's linked warehouse. This reuses the existing `inventory.StockMovements` infrastructure. Offline orders deduct on sync, not at sale time.

10.2 Stock Deduction Flow

POS Order Completed



PosOrderService.CompleteOrderAsync()



— 1. Validate all payments cover grand_total

— 2. For each PosOrderLine:



— PosStockService.DeductStockAsync(
 warehouseId: store.WarehouseId,
 productId: line.ProductId,
 quantity: line.Quantity,
 reference: \$"POS-{order.OrderNumber}",
 movementType: StockMovementType.PosSale // New enum value
)



— Creates inventory.StockMovements record:
 movement_type = 'POS_SALE'
 direction = 'OUT'
 warehouse_id = store's warehouse
 quantity = line quantity
 reference_type = 'PosOrder'
 reference_id = order_id

— 3. Update shift running totals

— 4. Update cashier session totals

— 5. If customer_id IS NOT NULL and payment method = Credit:

 Update customers.Customers.current_balance += order balance_due

10.3 Negative Stock Handling

POS allows sales when `stock_level ≤ 0` (negative stock).

Rationale: Retail cannot refuse a customer holding merchandise at the counter because the system count is wrong. Physical inventory takes precedence.

Behavior:

- Stock deduction proceeds even if available quantity would go negative.
- StockMovement is created regardless.
- A system notification is generated: "Product {SKU} has negative stock at warehouse {name}: {quantity}. Physical count recommended."
- The POS UI shows a ⚠ warning icon next to the stock badge on the product tile but does NOT block the sale.
- A Hangfire daily job generates a "Negative Stock Report" for all stores.

10.4 Return Stock Flow

POS Return Order Completed



For each return line (`is_return_line = true`):



```
└─ PosStockService.ReturnStockAsync(  
    warehouseId, productId, quantity,  
    reference: $"RET-{return_order.OrderNumber}",  
    movementType: StockMovementType.PosReturn // New enum value  
)
```



```
└─ Creates inventory.StockMovements:  
    movement_type = 'POS_RETURN'  
    direction = 'IN'
```

11. C10 — Finance Integration (Journal Entries)

11.1 Rationale

Every completed POS order creates a journal entry in the finance module. POS uses a summarized posting model — one journal entry per order (not per line) — for performance at retail transaction volumes.

11.2 Journal Entry Template

POS Sale Order #POS-STR001-20261010-0042

Grand Total: \$150.00 (Subtotal \$135.00 + Tax \$15.00)

Payment: Cash \$100.00, Card \$50.00

Journal Entry:

Date: 2026-10-10			
Reference: POS-STR001-20261010-0042			
Source: POS Module			
Debit		Credit	
Cash Account	100.00		
Card Receivable	50.00		
		Sales Revenue	135.00
		Tax Payable	15.00

11.3 Account Mapping Configuration

New table added during store setup (not a separate migration — configured at application level):

Payment Method	Default Account Type	Description
Cash (0)	Cash Account	Store cash register GL account
Card (1)	Card Receivable	Clearing account for card processor settlement
MobileWallet (2)	Mobile Receivable	Clearing account for mobile wallet settlement
BankTransfer (3)	Bank Account	Direct bank receipt
Credit (4)	Accounts Receivable	Customer AR sub-ledger
Cheque (5)	Cheque Receivable	Clearing account for cheques

Revenue and Tax accounts are configured at the store level:

csharp

```

public class PosAccountMapping
{
    // Stored in pos.Stores as JSON or separate config table
    public long SalesRevenueAccountId { get; set; }
    public long TaxPayableAccountId { get; set; }
    public long CostOfGoodsSoldAccountId { get; set; } // Future: COGS on sale
    public Dictionary<PaymentMethod, long> PaymentAccountMap { get; set; }
}

```

11.4 Credit Sale Journal

Credit Sale Order #POS-STR001-20261010-0055
 Grand Total: \$200.00 (Subtotal \$180.00 + Tax \$20.00)
 Payment: On-Account to Customer "Sarah Mitchell"

Journal Entry:

Debit		Credit	
Accounts Receivable	200.00		
(Sub: Customer #45)		Sales Revenue	180.00
		Tax Payable	20.00

→ Customer #45 current_balance updated: 1200.00 → 1400.00

11.5 Return Journal (Reversal)

Return Order #POS-STR001-20261010-R003
 Return Total: \$50.00 (Subtotal \$45.00 + Tax \$5.00)
 Refund: Cash \$50.00

Journal Entry:

Debit		Credit	
Sales Revenue	45.00		
Tax Payable	5.00		
		Cash Account	50.00

12. C11 — SignalR Real-Time Hub

12.1 Rationale

POS terminals need to receive real-time updates for price changes, stock level changes (from other channels), configuration changes, and force-sync commands. SignalR provides a persistent WebSocket connection with automatic fallback to long-polling.

12.2 Hub Definition

csharp

```
[Authorize]
[RequiresModule("POS")]
public class PosHub : Hub
{
    // Groups: each terminal joins a group named "store_{storeId}"
    // and "terminal_{terminalId}"

    public async Task JoinStore(long storeId)
    {
        await Groups.AddToGroupAsync(Context.ConnectionId, $"store_{storeId}");
    }

    public async Task JoinTerminal(long terminalId)
    {
        await Groups.AddToGroupAsync(Context.ConnectionId, $"terminal_{terminalId}");
    }

    // Client → Server: heartbeat, sync status
    public async Task Heartbeat(long terminalId)
    {
        await _terminalService.UpdateLastSeenAsync(terminalId);
    }
}
```

12.3 Server-to-Client Events

Event	Group	Payload	Trigger
PriceChanged	store_{id}	{ productId, oldPrice, newPrice }	Product price updated in back-office
StockUpdated	store_{id}	{ productId, warehouseId, available }	Stock movement in linked warehouse
ProductUpdated	store_{id}	{ productId, changes }	Product details changed (name, tax, status)
ProductDeactivated	store_{id}	{ productId }	Product deactivated or is_sellable set to false
CustomerBalanceChanged	store_{id}	{ customerId, newBalance, creditLimit }	Customer balance updated from another channel
ConfigChanged	store_{id}	{ configType, data }	Store config changed (tax rate, receipt text)
ForceSync	terminal_{id}	{ syncType }	Admin triggers full or delta sync on a terminal
ShiftAlert	terminal_{id}	{ message, severity }	Supervisor message to terminal
ForceLogout	terminal_{id}	{ reason }	Admin forces terminal logout

12.4 Offline Resilience

When SignalR disconnects:

1. PWA sets internal state: `isOnline = false`
2. UI shows "Offline" badge in header
3. All transactions continue using local data
4. Sync engine queues outbound data

When SignalR reconnects:

1. PWA sets `isOnline = true`
2. PWA triggers delta sync immediately (pull changes since last sync)
3. Sync engine pushes queued transactions
4. UI clears "Offline" badge
5. Hub re-joins store and terminal groups

13. C12 — PWA Client Architecture

13.1 Rationale

The POS client is a React 18 Progressive Web App deployed at `/pos` on the SMS web host. It installs on tablets and PCs, runs offline via Service Worker caching, and stores all data locally in IndexedDB via Dexie.js.

13.2 Technology Stack

Layer	Technology	Purpose
UI Framework	React 18 + Vite	Component rendering
State Management	React Context + useReducer	Cart state, auth state, sync state
Local Database	Dexie.js (IndexedDB wrapper)	Offline data persistence
Offline Cache	Service Worker (Workbox)	App shell caching, API fallback
Real-Time	@microsoft/signalr	WebSocket for live updates
HTTP Client	Axios	API calls with retry + queue
Barcode	quagga2 or html5-qrcode	Camera-based barcode scanning
Receipt	HTML template → <code>window.print()</code> or download	Digital receipt generation

13.3 Dexie.js Schema

typescript

```
import Dexie, { Table } from 'dexie';
```

```
interface LocalProduct {  
  productId: number;  
  name: string;  
  sku: string;  
  barcode?: string;  
  posName?: string;  
  salePrice: number;  
  basePrice: number;  
  categoryId?: number;  
  categoryName?: string;  
  taxRateId?: number;  
  taxPercentage: number;  
  color?: string;  
  imageUrl?: string;  
  isWeighable: boolean;  
  isFavorite: boolean;  
  sortOrder: number;  
  stockAvailable: number;  
  uomId: number;  
  uomName: string;  
  modifiedAt: string;  
}
```

```
interface LocalCustomer {  
  customerId: number;  
  code: string;  
  name: string;  
  customerType: number;  
  phone?: string;  
  email?: string;  
  creditLimit: number;  
  currentBalance: number;  
  allowPosCredit: boolean;  
  offlineCreditLimit?: number;  
  maxSingleCreditOrder?: number;  
  modifiedAt: string;  
}
```

```
interface LocalOrder {
```

```

localId: string;          // UUID primary key
orderNumber?: string;     // assigned on sync
serverId?: number;       // assigned on sync
storeId: number;
terminalId: number;
shiftId: number;
sessionId: number;
customerId?: number;
customerName?: string;
orderType: number;
status: number;
lines: LocalOrderLine[];
payments: LocalPayment[];
subtotal: number;
discountType: number;
discountValue: number;
discountAmount: number;
taxTotal: number;
grandTotal: number;
paidAmount: number;
changeAmount: number;
balanceDue: number;
paymentStatus: number;
isOffline: boolean;
syncStatus: 'pending' | 'syncing' | 'synced' | 'error';
syncError?: string;
createdAt: string;
createdBy: number;
}

// Dexie database definition
class PosDatabase extends Dexie {
  products!: Table<LocalProduct>;
  customers!: Table<LocalCustomer>;
  orders!: Table<LocalOrder>;
  config!: Table<{ key: string; value: any }>;
  syncLog!: Table<{ id?: number; type: string; timestamp: string; status: string }>;

  constructor() {
    super('sms-pos');
    this.version(1).stores({
      products: 'productId, barcode, categoryId, *name, isFavorite, modifiedAt',

```

```
        customers: 'customerId, code, *name, *phone, modifiedAt',
        orders: 'localId, serverId, shiftId, sessionId, syncStatus, createdAt',
        config: 'key',
        syncLog: '++id, type, timestamp'
    });
}
}

export const posDb = new PosDatabase();
```

13.4 Sync Engine

typescript

```

class SyncEngine {
  private isOnline: boolean = navigator.onLine;
  private syncInProgress: boolean = false;
  private lastSyncTimestamp: string | null = null;

  /**
   * PULL: Fetch changes from server since last sync.
   * Runs on: app launch, reconnect, SignalR reconnect,
   *          periodic interval (5 min when online).
   */
  async pullChanges(): Promise<void> {
    const since = this.lastSyncTimestamp || '1970-01-01T00:00:00Z';

    // Parallel fetch: catalog + customers
    const [catalog, customers] = await Promise.all([
      api.get(`/api/sync/catalog?since=${since}`),
      api.get(`/api/sync/customers?since=${since}`)
    ]);

    // Bulk upsert into IndexedDB
    await posDb.transaction('rw', posDb.products, posDb.customers, async () => {
      if (catalog.data.products.length > 0) {
        await posDb.products.bulkPut(
          catalog.data.products.map(mapServerProductToLocal)
        );
      }
      if (customers.data.customers.length > 0) {
        await posDb.customers.bulkPut(
          customers.data.customers.map(mapServerCustomerToLocal)
        );
      }
    });

    this.lastSyncTimestamp = catalog.data.syncTimestamp;
    await posDb.config.put({ key: 'lastSyncTimestamp', value: this.lastSyncTimestamp });
  }

  /**
   * PUSH: Upload pending orders and cash movements to server.
   * Runs on: order completion (if online), reconnect, periodic.
   */
}

```

```

async pushChanges(): Promise<void> {
  const pendingOrders = await posDb.orders
    .where('syncStatus').equals('pending')
    .toArray();

  for (const order of pendingOrders) {
    try {
      await posDb.orders.update(order.localId, { syncStatus: 'syncing' });

      const response = await api.post('/api/pos/orders/sync', {
        localId: order.localId,
        ...mapLocalOrderToServer(order)
      });

      await posDb.orders.update(order.localId, {
        syncStatus: 'synced',
        serverId: response.data.orderId,
        orderNumber: response.data.orderNumber
      });
    } catch (error) {
      await posDb.orders.update(order.localId, {
        syncStatus: 'error',
        syncError: error.message
      });
    }
  }
}

```

```

/**
 * Full sync: pulls everything (ignores since), then pushes.
 * Used on: first terminal activation, admin force-sync.
 */

```

```

async fullSync(): Promise<void> {
  this.lastSyncTimestamp = null;
  await posDb.products.clear();
  await posDb.customers.clear();
  await this.pullChanges();
  await this.pushChanges();
}

```

13.5 Service Worker Strategy

App Shell (Precache – installed with PWA):

- └─ index.html
- └─ main.{hash}.js
- └─ main.{hash}.css
- └─ manifest.json
- └─ static assets (icons, fonts)

API Caching Strategy:

- └─ /api/sync/* → Network-first, fallback to IndexedDB (via Dexie)
- └─ /api/pos/orders/* → Network-first, queue on failure (Background Sync API)
- └─ /api/pos/shifts/* → Network-first, fallback to cached
- └─ /api/auth/pos-pin → Network-first, fallback to cached PIN hashes

Offline Capability:

- └─ Create orders ✓ (stored in IndexedDB, synced later)
- └─ Browse products ✓ (cached in IndexedDB)
- └─ Search customers ✓ (cached in IndexedDB)
- └─ Process payments ✓ (recorded locally)
- └─ Cash movements ✓ (recorded locally)
- └─ Open new shift ✗ (requires supervisor online auth)
- └─ Close shift ⚠ (can enter count; final close requires sync)
- └─ Create customer ⚠ (created locally, synced later, no code until sync)
- └─ Returns ⚠ (allowed if original order is in local DB)

14. Database Migrations

Migration	Description	Schema	Depends On
M038.1	Create <code>pos</code> schema	pos	—
M038.2	Create <code>pos.Stores</code> table	pos	M038.1
M038.3	Create <code>pos.Terminals</code> table	pos	M038.2
M038.4	Create <code>pos.Shifts</code> table	pos	M038.3
M038.5	Create <code>pos.CashMovements</code> table	pos	M038.4
M038.6	Add <code>pos_pin</code> , <code>pos_role</code> , <code>default_store_id</code> to <code>auth.Users</code>	auth	M038.2
M038.7	Create <code>pos.CashierSessions</code> table	pos	M038.4, M038.6
M038.8	Create <code>pos.PosOrders</code> table	pos	M038.7
M038.9	Create <code>pos.PosOrderLines</code> table	pos	M038.8
M038.10	Create <code>pos.PosPayments</code> table	pos	M038.8
M038.11	Create <code>pos.ProductPosSettings</code> extension table	pos	M038.1
M038.12	Create <code>pos.CustomerPosSettings</code> table	pos	M038.1
M038.13	Add <code>POS_SALE</code> and <code>POS_RETURN</code> to <code>StockMovementType</code> enum	inventory	—
M038.14	Update <code>tenant.Modules</code> — set POS <code>is_available = 1</code>	tenant	M038.1– M038.12
M038.15	Seed POS module features in <code>tenant.ModuleFeatures</code>	tenant	M038.14
M038.16	Create <code>modified_at</code> triggers on POS tables	pos	M038.2– M038.12
M038.17	Seed POS permissions in <code>auth.Permissions</code>	auth	M038.14
M038.18	Add POS routes to <code>inventory.Routes</code>	inventory	M038.14

Migration	Description	Schema	Depends On
M038.19	Create POS-specific indexes	pos	M038.8, M038.9
M038.20	Seed POS workflow approval rules	workflow	M038.14

Migration SQL — Key Migrations

M038.1 — POS Schema

sql

CREATE SCHEMA pos;

M038.2 — Stores Table

sql

```

CREATE TABLE pos.Stores (
    store_id          BIGINT IDENTITY(1,1) PRIMARY KEY,
    org_id            BIGINT          NOT NULL,
    code              VARCHAR(20)     NOT NULL,
    name              NVARCHAR(200)   NOT NULL,
    warehouse_id      BIGINT          NOT NULL,
    address_line1     NVARCHAR(200)   NULL,
    address_line2     NVARCHAR(200)   NULL,
    city              NVARCHAR(100)   NULL,
    state_province    NVARCHAR(100)   NULL,
    postal_code       VARCHAR(20)     NULL,
    country_code      VARCHAR(3)      NULL,
    phone             VARCHAR(30)     NULL,
    email             NVARCHAR(200)   NULL,
    timezone          VARCHAR(50)     NOT NULL DEFAULT 'UTC',
    currency_code     VARCHAR(3)      NOT NULL DEFAULT 'USD',
    tax_inclusive_pricing BIT          NOT NULL DEFAULT 0,
    default_tax_rate_id BIGINT          NULL,
    receipt_header    NVARCHAR(500)   NULL,
    receipt_footer    NVARCHAR(500)   NULL,
    offline_credit_tolerance_pct DECIMAL(5,2) NOT NULL DEFAULT 10.00,
    auto_close_shift_hours INT          NOT NULL DEFAULT 24,
    is_active         BIT              NOT NULL DEFAULT 1,
    created_at        DATETIME2(2)     NOT NULL DEFAULT SYSUTCDATETIME(),
    created_by        BIGINT          NOT NULL,
    modified_at       DATETIME2(2)     NOT NULL DEFAULT SYSUTCDATETIME(),
    modified_by       BIGINT          NULL,
    row_version       ROWVERSION       NOT NULL,
    CONSTRAINT UQ_Stores_OrgCode UNIQUE (org_id, code),
    CONSTRAINT FK_Stores_Org FOREIGN KEY (org_id) REFERENCES tenant.Organizations(org_id)
    CONSTRAINT FK_Stores_Warehouse FOREIGN KEY (warehouse_id) REFERENCES inventory.Wareho
    CONSTRAINT FK_Stores_TaxRate FOREIGN KEY (default_tax_rate_id) REFERENCES lookups.Tax
    CONSTRAINT FK_Stores_CreatedBy FOREIGN KEY (created_by) REFERENCES auth.Users(user_id
);

CREATE INDEX IX_Stores_OrgActive ON pos.Stores (org_id, is_active);
CREATE INDEX IX_Stores_OrgWarehouse ON pos.Stores (org_id, warehouse_id);
CREATE INDEX IX_Stores_OrgModified ON pos.Stores (org_id, modified_at);

```

M038.4 — Shifts Table

sql

```

CREATE TABLE pos.Shifts (
    shift_id          BIGINT IDENTITY(1,1) PRIMARY KEY,
    org_id            BIGINT          NOT NULL,
    store_id          BIGINT          NOT NULL,
    terminal_id       BIGINT          NOT NULL,
    shift_number      VARCHAR(30)     NOT NULL,
    status            TINYINT         NOT NULL DEFAULT 0,
    opened_by        BIGINT          NOT NULL,
    opened_at         DATETIME2(2)    NULL,
    closed_by        BIGINT          NULL,
    closed_at         DATETIME2(2)    NULL,
    reconciled_by    BIGINT          NULL,
    reconciled_at     DATETIME2(2)    NULL,
    opening_float     DECIMAL(18,4)   NOT NULL DEFAULT 0,
    total_cash_sales  DECIMAL(18,4)   NOT NULL DEFAULT 0,
    total_card_sales  DECIMAL(18,4)   NOT NULL DEFAULT 0,
    total_mobile_sales DECIMAL(18,4)   NOT NULL DEFAULT 0,
    total_credit_sales DECIMAL(18,4)   NOT NULL DEFAULT 0,
    total_cash_refunds DECIMAL(18,4)   NOT NULL DEFAULT 0,
    total_card_refunds DECIMAL(18,4)   NOT NULL DEFAULT 0,
    total_cash_in     DECIMAL(18,4)   NOT NULL DEFAULT 0,
    total_cash_out     DECIMAL(18,4)   NOT NULL DEFAULT 0,
    expected_cash_closing DECIMAL(18,4) NOT NULL DEFAULT 0,
    actual_cash_closing DECIMAL(18,4)   NULL,
    cash_variance     DECIMAL(18,4)   NULL,
    variance_threshold DECIMAL(18,4)   NOT NULL DEFAULT 5.00,
    variance_note     NVARCHAR(500)   NULL,
    order_count       INT             NOT NULL DEFAULT 0,
    refund_count      INT             NOT NULL DEFAULT 0,
    gross_total       DECIMAL(18,4)   NOT NULL DEFAULT 0,
    net_total         DECIMAL(18,4)   NOT NULL DEFAULT 0,
    is_synced         BIT             NOT NULL DEFAULT 0,
    created_at        DATETIME2(2)    NOT NULL DEFAULT SYSUTCDATETIME(),
    modified_at       DATETIME2(2)    NOT NULL DEFAULT SYSUTCDATETIME(),
    row_version       ROWVERSION      NOT NULL,
    CONSTRAINT UQ_Shifts_OrgNumber UNIQUE (org_id, shift_number),
    CONSTRAINT FK_Shifts_Org FOREIGN KEY (org_id) REFERENCES tenant.Organizations(org_id),
    CONSTRAINT FK_Shifts_Store FOREIGN KEY (store_id) REFERENCES pos.Stores(store_id),
    CONSTRAINT FK_Shifts_Terminal FOREIGN KEY (terminal_id) REFERENCES pos.Terminals(term
    CONSTRAINT FK_Shifts_OpenedBy FOREIGN KEY (opened_by) REFERENCES auth.Users(user_id),
    CONSTRAINT CK_Shifts_Status CHECK (status IN (0,1,2,3,4,5))

```

```
);
```

```
CREATE INDEX IX_Shifts_TerminalStatus ON pos.Shifts (terminal_id, status);
```

```
CREATE INDEX IX_Shifts_StoreStatus ON pos.Shifts (store_id, status);
```

```
CREATE INDEX IX_Shifts_OrgOpened ON pos.Shifts (org_id, opened_at);
```

M038.8 — POS Orders Table

```
sql
```

```

CREATE TABLE pos.PosOrders (
    order_id          BIGINT IDENTITY(1,1) PRIMARY KEY,
    org_id            BIGINT          NOT NULL,
    store_id          BIGINT          NOT NULL,
    terminal_id       BIGINT          NOT NULL,
    shift_id          BIGINT          NOT NULL,
    session_id        BIGINT          NOT NULL,
    order_number      VARCHAR(30)     NOT NULL,
    order_type        TINYINT         NOT NULL DEFAULT 0,
    status            TINYINT         NOT NULL DEFAULT 0,
    customer_id       BIGINT          NULL,
    customer_name     NVARCHAR(200)   NULL,
    line_count        INT             NOT NULL DEFAULT 0,
    subtotal          DECIMAL(18,4)   NOT NULL DEFAULT 0,
    discount_type     TINYINT         NOT NULL DEFAULT 0,
    discount_value    DECIMAL(18,4)   NOT NULL DEFAULT 0,
    discount_amount   DECIMAL(18,4)   NOT NULL DEFAULT 0,
    tax_total         DECIMAL(18,4)   NOT NULL DEFAULT 0,
    grand_total       DECIMAL(18,4)   NOT NULL DEFAULT 0,
    paid_amount       DECIMAL(18,4)   NOT NULL DEFAULT 0,
    change_amount     DECIMAL(18,4)   NOT NULL DEFAULT 0,
    balance_due       DECIMAL(18,4)   NOT NULL DEFAULT 0,
    payment_status    TINYINT         NOT NULL DEFAULT 0,
    original_order_id BIGINT          NULL,
    return_reason     NVARCHAR(300)   NULL,
    note             NVARCHAR(500)   NULL,
    local_id         VARCHAR(50)     NOT NULL,
    is_offline        BIT             NOT NULL DEFAULT 0,
    synced_at        DATETIME2(2)     NULL,
    created_at        DATETIME2(2)     NOT NULL DEFAULT SYSUTCDATETIME(),
    created_by        BIGINT          NOT NULL,
    modified_at       DATETIME2(2)     NOT NULL DEFAULT SYSUTCDATETIME(),
    row_version       ROWVERSION      NOT NULL,
    CONSTRAINT UQ_PosOrders_OrgNumber UNIQUE (org_id, order_number),
    CONSTRAINT UQ_PosOrders_OrgLocalId UNIQUE (org_id, local_id),
    CONSTRAINT FK_PosOrders_Org FOREIGN KEY (org_id) REFERENCES tenant.Organizations(org_id),
    CONSTRAINT FK_PosOrders_Store FOREIGN KEY (store_id) REFERENCES pos.Stores(store_id),
    CONSTRAINT FK_PosOrders_Terminal FOREIGN KEY (terminal_id) REFERENCES pos.Terminals(terminal_id),
    CONSTRAINT FK_PosOrders_Shift FOREIGN KEY (shift_id) REFERENCES pos.Shifts(shift_id),
    CONSTRAINT FK_PosOrders_Session FOREIGN KEY (session_id) REFERENCES pos.CashierSessions(session_id),
    CONSTRAINT FK_PosOrders_Customer FOREIGN KEY (customer_id) REFERENCES customers.Customers(customer_id)
);

```

```

CONSTRAINT FK_PosOrders_OriginalOrder FOREIGN KEY (original_order_id) REFERENCES pos.
CONSTRAINT CK_PosOrders_Status CHECK (status IN (0,1,2,3,4)),
CONSTRAINT CK_PosOrders_Type CHECK (order_type IN (0,1,2))
);

CREATE INDEX IX_PosOrders_StoreDate ON pos.PosOrders (store_id, created_at);
CREATE INDEX IX_PosOrders_ShiftStatus ON pos.PosOrders (shift_id, status);
CREATE INDEX IX_PosOrders_Session ON pos.PosOrders (session_id);
CREATE INDEX IX_PosOrders_Customer ON pos.PosOrders (customer_id) WHERE customer_id IS NOT NULL;
CREATE INDEX IX_PosOrders_PendingSync ON pos.PosOrders (org_id, synced_at) WHERE synced_at IS NOT NULL;

```

M038.14 — Activate POS Module

```

sql

-- Set POS module as available
UPDATE tenant.Modules SET is_available = 1 WHERE code = 'POS';

```

M038.15 — POS Module Features

```

sql

INSERT INTO tenant.ModuleFeatures (module_id, code, name, description, is_core_feature, is_available, seq)
SELECT m.module_id, f.code, f.name, f.description, f.is_core, f.is_avail, f.seq
FROM tenant.Modules m
CROSS APPLY (VALUES
    ('POS_ORDERS', 'POS Order Processing', 'Create and manage POS sale order'),
    ('POS_SHIFTS', 'Shift Management', 'Open/close/reconcile shifts with'),
    ('POS_CASHIER_PIN', 'Cashier PIN Authentication', 'Shared terminal PIN-based cashier login'),
    ('POS_PAYMENTS', 'Payment Recording', 'Record cash, card, mobile payments'),
    ('POS_OFFLINE', 'Offline Mode', 'Process transactions without internet'),
    ('POS_RETURNS', 'POS Returns & Refunds', 'Process returns and refunds at POS'),
    ('POS_CREDIT_SALES', 'Credit Sales (On-Account)', 'Allow credit sales to registered customers'),
    ('POS_BARCODE', 'Barcode Scanning', 'Camera or scanner based barcode scanning'),
    ('POS_MULTI_STORE', 'Multi-Store Management', 'Manage multiple retail locations'),
    ('POS_ANALYTICS', 'POS Analytics Dashboard', 'Sales analytics and reporting'),
    ('POS_PROMOTIONS', 'Promotions Engine', 'Advanced promotions and coupon management'),
    ('POS_LOYALTY', 'Loyalty Integration', 'Customer loyalty points at POS')
) AS f(code, name, description, is_core, is_avail, seq)
WHERE m.code = 'POS';

```

M038.17 — POS Permissions

sql

```
INSERT INTO auth.Permissions (code, name, module_code, description)
VALUES
    ('pos.store.view', 'View Stores', 'POS', 'View store list and details'),
    ('pos.store.manage', 'Manage Stores', 'POS', 'Create, update, deactivate store'),
    ('pos.terminal.view', 'View Terminals', 'POS', 'View terminal list'),
    ('pos.terminal.manage', 'Manage Terminals', 'POS', 'Create, update, bind terminal'),
    ('pos.shift.open', 'Open Shift', 'POS', 'Open a new shift on a terminal'),
    ('pos.shift.close', 'Close Shift', 'POS', 'Close and count a shift'),
    ('pos.shift.reconcile', 'Reconcile Shift', 'POS', 'Reconcile shift variables'),
    ('pos.shift.view', 'View Shifts', 'POS', 'View shift history and details'),
    ('pos.order.create', 'Create POS Order', 'POS', 'Process sales at POS terminal'),
    ('pos.order.view', 'View POS Orders', 'POS', 'View POS order history and details'),
    ('pos.order.void', 'Void POS Order', 'POS', 'Void a completed POS order'),
    ('pos.order.return', 'Process Return', 'POS', 'Process returns and refunds'),
    ('pos.order.discount', 'Apply Discount', 'POS', 'Apply line or order level discount'),
    ('pos.cash.movement', 'Cash In/Out', 'POS', 'Record cash movements'),
    ('pos.product.pos_settings', 'Manage POS Product Settings', 'POS', 'Configure POS-specific product settings'),
    ('pos.customer.pos_settings', 'Manage POS Customer Settings', 'POS', 'Configure POS-specific customer settings'),
    ('pos.reports.view', 'View POS Reports', 'POS', 'Access POS reporting dashboard')
```

15. API Endpoints

15.1 Store Management API

#	Method	Endpoint	Auth	Description
1	GET	<code>/api/pos/stores</code>	pos.store.view	List stores for current tenant
2	GET	<code>/api/pos/stores/{id}</code>	pos.store.view	Get store detail
3	POST	<code>/api/pos/stores</code>	pos.store.manage	Create store
4	PUT	<code>/api/pos/stores/{id}</code>	pos.store.manage	Update store
5	PATCH	<code>/api/pos/stores/{id}/status</code>	pos.store.manage	Activate/deactivate store

15.2 Terminal API

#	Method	Endpoint	Auth	Description
6	GET	<code>/api/pos/stores/{storeId}/terminals</code>	pos.terminal.view	List terminals in store
7	POST	<code>/api/pos/stores/{storeId}/terminals</code>	pos.terminal.manage	Create terminal
8	PUT	<code>/api/pos/terminals/{id}</code>	pos.terminal.manage	Update terminal
9	POST	<code>/api/pos/terminals/{id}/bind</code>	pos.terminal.manage	Bind device UUID
10	POST	<code>/api/pos/terminals/{id}/unbind</code>	pos.terminal.manage	Unbind device
11	GET	<code>/api/pos/terminals/{id}/status</code>	pos.terminal.view	Terminal connectivity status

15.3 Shift API

#	Method	Endpoint	Auth	Description
12	POST	<code>/api/pos/terminals/{terminalId}/shifts</code>	pos.shift.open	Open new shift
13	GET	<code>/api/pos/shifts/{id}</code>	pos.shift.view	Get shift details + running totals
14	PATCH	<code>/api/pos/shifts/{id}/close</code>	pos.shift.close	Submit actual closing count
15	PATCH	<code>/api/pos/shifts/{id}/reconcile</code>	pos.shift.reconcile	Reconcile or dispute shift
16	GET	<code>/api/pos/stores/{storeId}/shifts</code>	pos.shift.view	List shifts by store + date range
17	GET	<code>/api/pos/shifts/{id}/summary</code>	pos.shift.view	Shift summary (totals by method, order count)

15.4 Cashier Session API

#	Method	Endpoint	Auth	Description
18	POST	<code>/api/pos/auth/pin</code>	—	Verify cashier PIN (returns user context + permissions)
19	POST	<code>/api/pos/sessions</code>	pos.order.create	Start/resume cashier session
20	PATCH	<code>/api/pos/sessions/{id}/end</code>	pos.order.create	End cashier session

15.5 POS Order API

#	Method	Endpoint	Auth	Description
21	POST	<code>/api/pos/orders/sync</code>	pos.order.create	Sync order from PWA (create or update)
22	POST	<code>/api/pos/orders/sync/batch</code>	pos.order.create	Batch sync multiple orders
23	GET	<code>/api/pos/orders/{id}</code>	pos.order.view	Get order detail
24	GET	<code>/api/pos/orders?storeId=&date=</code>	pos.order.view	List orders (paginated)
25	PATCH	<code>/api/pos/orders/{id}/void</code>	pos.order.void	Void a completed order
26	POST	<code>/api/pos/orders/{id}/return</code>	pos.order.return	Create return order from original

15.6 Cash Movement API

#	Method	Endpoint	Auth	Description
27	POST	<code>/api/pos/shifts/{shiftId}/cash-movements</code>	pos.cash.movement	Record cash in/out
28	GET	<code>/api/pos/shifts/{shiftId}/cash-movements</code>	pos.shift.view	List cash movements for shift

15.7 POS Sync API (Extensions to Addendum 37)

#	Method	Endpoint	Auth	Description
29	GET	<code>/api/sync/catalog?since={iso}</code>	pos.order.create	Extended: includes posSettings + stock for store warehouse
30	GET	<code>/api/sync/customers?since={iso}</code>	pos.order.create	Extended: includes posSettings
31	GET	<code>/api/sync/pos-config?storeId={id}</code>	pos.order.create	Store config, terminal config, payment methods, tax rates
32	GET	<code>/api/sync/pos-users?storeId={id}</code>	pos.order.create	Active POS users for store with hashed PINs (offline auth)

15.8 POS Product / Customer Settings API

#	Method	Endpoint	Auth	Description
33	GET	<code>/api/products/{id}/pos-settings</code>	pos.product.pos_settings	Get POS settings for product
34	PUT	<code>/api/products/{id}/pos-settings</code>	pos.product.pos_settings	Update POS settings
35	POST	<code>/api/products/pos-settings/bulk</code>	pos.product.pos_settings	Bulk update POS settings
36	GET	<code>/api/customers/{id}/pos-settings</code>	pos.customer.pos_settings	Get POS settings for customer
37	PUT	<code>/api/customers/{id}/pos-settings</code>	pos.customer.pos_settings	Update POS settings

All POS controllers tagged:

```
csharp
[ApiController]
[RequiresModule("POS")]
public class PosOrdersController : ControllerBase { }
```

16. Permission Claims

16.1 New Permissions

Code	Module	Description
<code>pos.store.view</code>	POS	View store list and details
<code>pos.store.manage</code>	POS	Create, update, deactivate stores
<code>pos.terminal.view</code>	POS	View terminal list
<code>pos.terminal.manage</code>	POS	Create, update, bind terminals
<code>pos.shift.open</code>	POS	Open a new shift on a terminal
<code>pos.shift.close</code>	POS	Close and count a shift
<code>pos.shift.reconcile</code>	POS	Reconcile shift variance (supervisor)
<code>pos.shift.view</code>	POS	View shift history and details
<code>pos.order.create</code>	POS	Process sales at POS terminal
<code>pos.order.view</code>	POS	View POS order history
<code>pos.order.void</code>	POS	Void a completed POS order
<code>pos.order.return</code>	POS	Process returns and refunds
<code>pos.order.discount</code>	POS	Apply line or order level discounts
<code>pos.cash.movement</code>	POS	Record cash movements during shift
<code>pos.product.pos_settings</code>	POS	Configure POS-specific product attributes
<code>pos.customer.pos_settings</code>	POS	Configure POS credit and customer settings
<code>pos.reports.view</code>	POS	Access POS reporting and analytics

16.2 Default Role Mapping

Permission	Cashier	Supervisor	Store Manager	Admin
pos.order.create	✓	✓	✓	✓
pos.order.view	Own only	✓	✓	✓
pos.order.discount	✗	✓	✓	✓
pos.order.void	✗	✓	✓	✓
pos.order.return	✗	✓	✓	✓
pos.shift.open	✗	✓	✓	✓
pos.shift.close	✗	✓	✓	✓
pos.shift.reconcile	✗	✗	✓	✓
pos.cash.movement	✗	✓	✓	✓
pos.store.manage	✗	✗	✗	✓
pos.terminal.manage	✗	✗	✓	✓

17. POS Screen Design & UI Specifications

17.1 POS Application Routes (PWA)

Route	Screen	Auth
/pos	Terminal selector / PIN entry	Device UUID match
/pos/sell	Main selling interface	Active cashier session
/pos/cart	Cart review (mobile layout)	Active cashier session
/pos/payment	Payment tendering	Active cashier session
/pos/receipt/{orderId}	Receipt view	Active cashier session
/pos/shift	Shift management	Supervisor
/pos/shift/close	Shift closing count	Supervisor
/pos/history	Order history search	Supervisor
/pos/settings	Terminal settings	Supervisor

17.2 Main Selling Screen (Desktop / Tablet Landscape)



Layout Specification:

Area	Width	Description
Product Grid (left)	60%	Scrollable product tiles with category tabs
Cart Panel (right)	40%	Current order lines, totals, pay button
Top Bar	100%	Store info, terminal, shift, cashier, connectivity status

Product Tile Specification:

Element	Detail
Size	~120×140px (responsive grid: auto-fill, min 110px)
Background	POS category color or product <code>color</code> from posSettings
Icon / Image	Product image if <code>image_url</code> set, else category icon
Product Name	<code>pos_name</code> if set, else product <code>name</code> (max 2 lines, ellipsis)
Price	<code>sale_price</code> if set, else product <code>base_price</code>
Stock Badge	Available qty from local cache.  icon if ≤ 0
Tap Action	Add 1 unit to cart (or open quantity input if <code>is_weighable</code>)
Long Press	Open product detail (full name, SKU, barcode, stock, price history)

Search Bar Behavior:

- Keyboard entry: instant filter on product name, SKU, barcode
- Barcode scanner input: auto-detected by rapid character entry + Enter suffix → direct add to cart
- No results: show "Product not found" with option to search by barcode scan

17.3 Payment Screen

← Back to Cart

Payment

Order Total

\$42.22

Remaining

\$42.22

Payments Tendered:

(none yet)

Select Payment Method



Cash



Card



Mobile



Transfer



On-Acct

Amount: \$

Quick amounts:

\$20

\$50

\$100

Exact

✓ CONFIRM PAYMENT

Order Total: \$42.22

Payments:

Cash \$30.00

Remaining: \$12.22

[+ Add Payment Method]

Card \$12.22 (exact remaining)

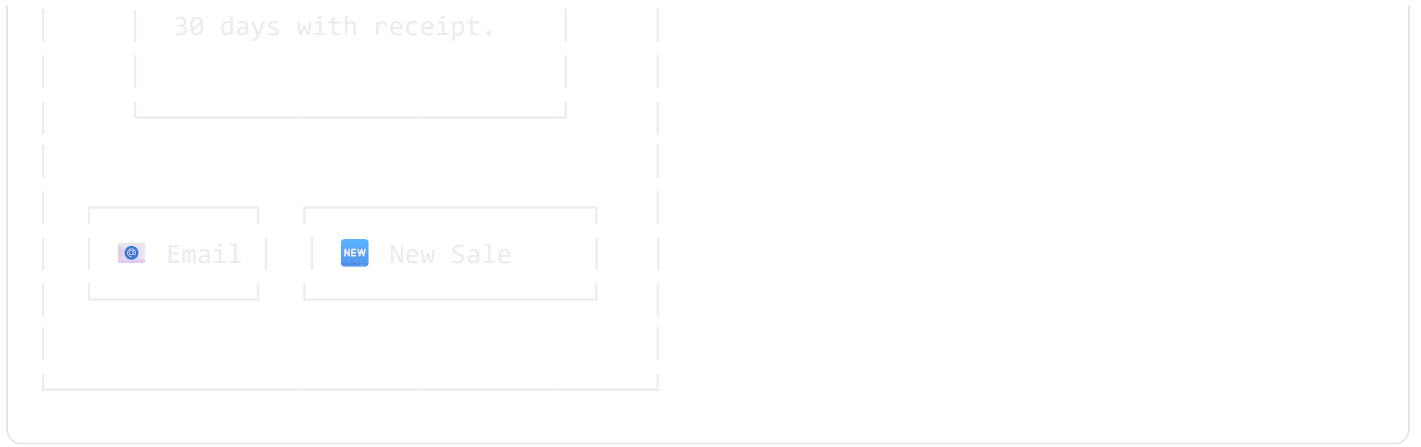
Remaining: \$0.00

[✓ Complete Sale]

On-Account (Credit) Payment:

1. Cashier selects "On-Account"
2. System checks:
 - a. Is customer assigned?
NO → "Assign a customer to use On-Account"
 - b. Is customer Walk-In?
YES → "Walk-In customers cannot use credit"
 - c. Check credit limit (online or cached):
 $\text{current_balance} + \text{order_total} > \text{credit_limit}?$
YES → "Credit limit exceeded. Limit: \$5,000. Balance: \$4,200. This order: \$900."
3. If allowed → Payment created with method = Credit (4)
4. Customer balance updated on order completion

17.4 Receipt Screen



Receipt data source: Receipt header and footer from `pos.Stores`. Line items from order.
Receipt is an HTML template rendered client-side.

17.5 Shift Management Screen (Supervisor)

← Back

Shift Management

Shift #SH-STR001-20261010-01

Status: OPEN

Terminal: Register 1

Opened: 2026-10-10 09:00 by Maria S.

Opening Float

\$200.00

Cash Sales

\$1,450.00

Card Sales

\$2,380.00

Mobile Sales

\$560.00

Cash In

\$50.00

Cash Out

\$300.00

Cash Refunds

\$25.00

Expected Cash

\$1,375.00

Cashier Sessions

Cashier	Started	Orders	Cash	Card	Status
Ahmed K.	09:05	23	\$580.00	\$920.00	 Active
Fatima R.	12:00	18	\$420.00	\$780.00	 Ended 14:30

James T.	14:30	12	\$450.00	\$680.00	Active
----------	-------	----	----------	----------	-------------------------------

Cash Movements

Time	Type	Amount	By	Reason
11:30	Cash Out	\$200.00	Maria S.	Safe Drop
13:15	Cash Out	\$100.00	Maria S.	Petty Cash
15:00	Cash In	\$50.00	Maria S.	Float Top-Up

Cash In	Cash Out	Close Shift
--------------------------------	---------------------------------	------------------------------------

17.6 Shift Closing Screen

Close Shift #SH-STR001-20261010-01

Expected Cash in Drawer: \$1,375.00

Count your cash and enter the amount:

Actual Cash: \$ [1,370.00]

Variance: -\$5.00

Threshold: \$5.00

Status: ☒ Within tolerance

Note (optional):

Shift Summary

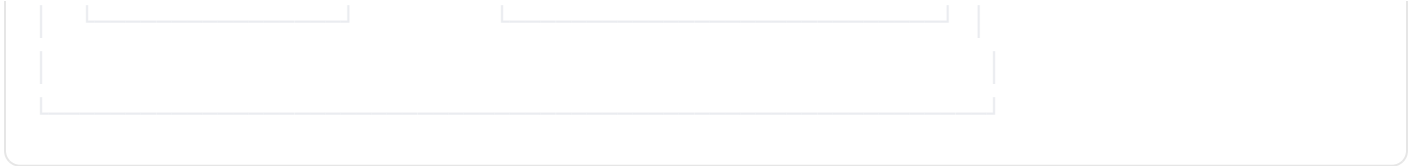
Total Orders:	53
Total Refunds:	2
Gross Sales:	\$4,490.00
Refunds:	-\$45.00
Net Sales:	\$4,445.00

By Payment Method:

Cash:	\$1,450.00
Card:	\$2,380.00
Mobile:	\$560.00
On-Account:	\$55.00

Cancel

✓ Submit & Close Shift



17.7 Back-Office POS Screens (SMS Admin)

17.7.1 Store List Page

Route: `/settings/pos/stores`

Column	Width	Sortable	Filterable
Code	100px	✓	✓
Name	200px	✓	✓ (search)
Warehouse	200px	✓	✓ (dropdown)
Terminals	80px	✓	✗
Active Shifts	80px	✗	✗
City	150px	✓	✓
Status	80px	✓	✓

17.7.2 Store Detail Page

Route: `/settings/pos/stores/{id}`

Tabs:

Tab	Content
General	Name, code, address, phone, email, timezone, currency
Configuration	Tax settings (inclusive/exclusive, default rate), receipt header/footer, offline credit tolerance, auto-close hours
Terminals	Terminal list with status, device binding, last seen
Staff	POS users assigned to this store (cashiers, supervisors) with PIN management
Shifts	Shift history with date filter, status filter, variance highlight
Reports	Store-level sales summary by day/week/month

17.7.3 Product POS Tab (Extension)

Added to Product form when POS module enabled and `is_sellable = true`:

POS Settings

Barcode:

[5901234567890]

[ Scan]

POS Display Name:

[Arabica 250g]

(optional)

Sale Price:

[14.99]

(optional)

POS Category:

[ Hot Drinks ▼]

Tile Color:

[ #6F4E37 ▼]

Image URL:

[/images/arabica.jpg]

☒ Available on POS

☒ Favorite (quick access)

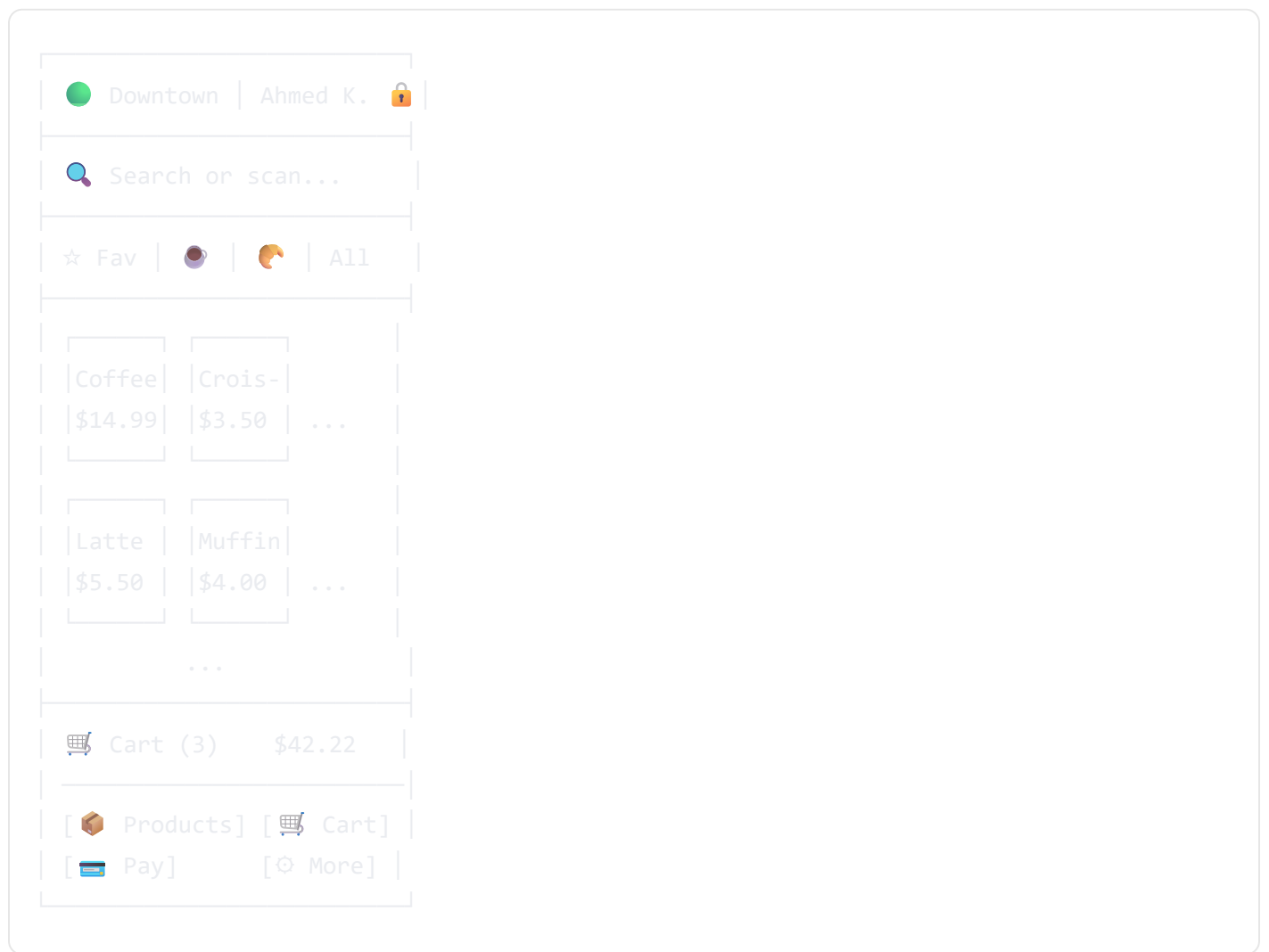
☐ Weighable item

Sort Order: [1]

[Save POS Settings]

17.8 Mobile Layout (Phone / Portrait Tablet)

On narrow viewports (< 768px), the selling screen switches to a single-column layout with bottom tab navigation:



Tapping " Cart" shows the full cart view. Tapping " Pay" goes to payment (only if cart has items).

18. Business Rules

18.1 Store Rules

Rule ID	Rule	Description
POS-STR-01	One warehouse per store	Each store links to exactly one warehouse. Validated at application level
POS-STR-02	Unique warehouse binding	A warehouse can be linked to at most one store. Attempting to bind a warehouse already linked to another store fails with conflict error
POS-STR-03	Store deactivation cascade	Deactivating a store deactivates all its terminals. Requires all shifts to be closed first
POS-STR-04	Currency from warehouse	Store currency must match the warehouse's organization currency
POS-STR-05	Timezone required	Store timezone determines shift date boundary for reporting and shift number generation

18.2 Terminal Rules

Rule ID	Rule	Description
POS-TRM-01	One active shift	A terminal can have at most one shift in status OPEN or CLOSING at any time
POS-TRM-02	Device binding optional	A terminal can operate without device binding (manual terminal selection at login)
POS-TRM-03	Device rebind	Unbinding and rebinding to a different device is a supervisor action
POS-TRM-04	Heartbeat stale	Terminal with <code>last_seen_at</code> > 15 minutes shows as "Offline" in admin

18.3 Shift Rules

Rule ID	Rule	Description
POS-SHF-01	Supervisor opens	Only users with <code>pos_role = Supervisor</code> or <code>pos.shift.open</code> permission can open a shift
POS-SHF-02	Opening float required	Opening float amount must be entered (can be 0) before shift transitions to OPEN
POS-SHF-03	No sales in DRAFT	Orders cannot be created until shift status = OPEN
POS-SHF-04	Close blocks new sales	When shift transitions to CLOSING, no new orders can be started. In-progress orders can be completed
POS-SHF-05	Expected cash auto-calculated	<code>expected_cash_closing</code> is recalculated on every cash-affecting event (sale, refund, cash movement)
POS-SHF-06	Variance auto-flag	If `
POS-SHF-07	Supervisor reconciles dispute	Only supervisor can resolve a DISPUTED shift (enter note, override to RECONCILED)
POS-SHF-08	Stale shift alert	Hangfire job checks shifts OPEN longer than <code>auto_close_shift_hours</code> → creates notification for store supervisor
POS-SHF-09	Shift number auto-gen	Format: <code>SH-{store_code}-{YYYYMMDD}-{seq}</code> . Date uses store timezone
POS-SHF-10	Closed shift immutable	After RECONCILED status, shift totals cannot be changed. Only <code>variance_note</code> can be appended

18.4 Cashier Rules

Rule ID	Rule	Description
POS-CSH-01	PIN unique per store	No two active users in the same store may share the same PIN. Validated on PIN set/update
POS-CSH-02	PIN length 4-6 digits	PIN must be between 4 and 6 numeric digits
POS-CSH-03	PIN hashed	PIN stored as bcrypt hash (cost factor 10). Never stored or logged in plaintext
POS-CSH-04	Session auto-end	When a different cashier enters their PIN, the previous cashier's session is ended automatically
POS-CSH-05	Cart cleared on switch	Switching cashiers clears any draft (incomplete) order. Completed orders are unaffected
POS-CSH-06	No PIN skip	If store has >1 active POS user, PIN entry is required. Single-cashier store may optionally skip PIN
POS-CSH-07	Failed PIN lockout	After 5 consecutive failed PIN attempts on a terminal, that terminal locks for 5 minutes. Supervisor can unlock
POS-CSH-08	Offline PIN verification	When offline, PIN verified against locally cached bcrypt hashes (synced at last sync)

18.5 Order Rules

Rule ID	Rule	Description
POS-ORD-01	Only sellable products	Only products with <code>is_sellable = true</code> AND <code>pos.ProductPosSettings.is_pos_active = true</code> appear in POS grid
POS-ORD-02	Price snapshot	Unit price captured at sale time cannot be retroactively changed
POS-ORD-03	Tax snapshot	Tax rate percentage captured at sale time
POS-ORD-04	Order total formula	<code>grand_total = subtotal - discount_amount + tax_total</code>
POS-ORD-05	Payment must cover	Order cannot transition to Completed unless <code>paid_amount >= grand_total</code> (or method = Credit)
POS-ORD-06	Void supervisor only	Only supervisor can void a completed order
POS-ORD-07	Void within shift	Orders can only be voided within the same shift they were created
POS-ORD-08	Return any shift	Returns can be processed against orders from any past shift
POS-ORD-09	Return max = original	Return quantity per line cannot exceed original order line quantity
POS-ORD-10	Offline order dedup	Backend uses <code>local_id</code> (UUID) to detect duplicate sync submissions. Second submission returns existing order
POS-ORD-11	Walk-in default	If no customer selected, order is assigned to the store's Walk-In customer (<code>customer_type = 0</code>)
POS-ORD-12	Negative stock allowed	Sales proceed even when stock level is zero or negative. Warning shown, stock movement recorded
POS-ORD-13	Stale price warning	If online and product price has changed since local cache, show warning: "Price updated: was \$X, now \$Y. Use new price?"

18.6 Payment Rules

Rule ID	Rule	Description
POS-PAY-01	Multi-tender allowed	An order can have multiple payments (split payment across methods)
POS-PAY-02	Cash change calculated	<code>change_given = paid_amount - remaining</code> when Cash method and amount > remaining
POS-PAY-03	Card no change	Card/Mobile/Transfer payments cannot exceed remaining balance
POS-PAY-04	Credit customer required	On-Account payment requires a non-Walk-In customer assigned to the order
POS-PAY-05	Credit limit check (online)	Online: <code>customer.current_balance + order_total <= credit_limit</code>
POS-PAY-06	Credit limit check (offline)	Offline: <code>cached_balance + order_total <= cached_limit × (1 + tolerance_pct / 100)</code>
POS-PAY-07	Credit over-limit flag	Orders that exceed credit limit (discovered on sync) flagged as CREDIT_OVER_LIMIT for supervisor review
POS-PAY-08	Walk-in cash only offline	Walk-In customers can only pay Cash when terminal is offline
POS-PAY-09	Refund reversal	Refund payments create reverse journal entries and credit customer balance if was credit sale

18.7 Sync Rules

Rule ID	Rule	Description
POS-SYN-01	Delta sync interval	PWA pulls catalog/customer changes every 5 minutes when online
POS-SYN-02	Full sync on first load	First terminal activation performs full sync (all products, customers, config)
POS-SYN-03	Order push immediate	Completed orders pushed to server immediately if online, queued if offline
POS-SYN-04	Batch push on reconnect	On reconnect, all pending orders pushed in chronological batch
POS-SYN-05	Idempotent sync	All sync endpoints use <code>local_id</code> for idempotent processing. Retry-safe
POS-SYN-06	Conflict resolution	Server is source of truth. If server has a newer version of a record, server wins
POS-SYN-07	Stale data warning	Products not synced in >24 hours show a "Data may be outdated" banner

19. Test Scenarios

ID	Scenario	Expected Result
Store & Terminal		
TS-038-01	Create store linked to warehouse WH-001	Store created with 1:1 warehouse binding
TS-038-02	Create second store linked to WH-001	Fail — "Warehouse already linked to another store"
TS-038-03	Create terminal T-01 in store STR-001	Terminal created with device_uuid = NULL
TS-038-04	Bind device to terminal (POST /bind with UUID)	device_uuid set, last_seen_at updated
TS-038-05	Deactivate store with open shift	Fail — "Close all shifts before deactivating store"
Shift Lifecycle		
TS-038-06	Open shift with \$200 float (supervisor)	Shift created: status=OPEN, opening_float=200
TS-038-07	Open shift (cashier, no supervisor permission)	Fail — 403 Forbidden
TS-038-08	Open second shift on same terminal	Fail — "Terminal already has an active shift"
TS-038-09	Process 3 cash sales (\$50, \$30, \$20) during shift	total_cash_sales = \$100, expected_cash = \$300
TS-038-10	Cash-out \$100 (safe drop) during shift	total_cash_out = \$100, expected_cash = \$200
TS-038-11	Close shift, enter actual \$195 (variance −\$5, threshold \$5)	Status → RECONCILED (within tolerance)
TS-038-12	Close shift, enter actual \$180 (variance −\$20, threshold \$5)	Status → DISPUTED (exceeds tolerance)

ID	Scenario	Expected Result
TS-038-13	Supervisor resolves disputed shift with note	Status → RECONCILED, variance_note saved
TS-038-14	Shift open > 24 hours (auto_close_shift_hours)	Hangfire job creates stale shift notification
Cashier PIN		
TS-038-15	Set PIN 1234 for cashier Ahmed in store STR-001	PIN hash saved, pos_role = Cashier
TS-038-16	Set PIN 1234 for cashier Fatima in same store	Fail — "PIN already in use at this store"
TS-038-17	Enter correct PIN at terminal	Cashier session created, cart ready
TS-038-18	Enter wrong PIN 5 times	Terminal locked for 5 minutes
TS-038-19	Switch cashier (different PIN entered)	Previous session ended, new session created, cart cleared
TS-038-20	Enter PIN offline (terminal disconnected)	Verified against cached PIN hash — session starts
Order Processing		
TS-038-21	Add 2× Arabica Coffee (\$14.99) to cart	Cart shows: $2 \times \$14.99 = \29.98
TS-038-22	Apply 10% line discount	Line discount = \$3.00, line total = \$26.98
TS-038-23	Apply 5% order discount	Order discount calculated on subtotal
TS-038-24	Pay \$50 cash for \$42.22 order	Change \$7.78, order Completed, shift cash updated
TS-038-25	Split payment: \$30 cash + \$12.22 card	Two PosPayment records, order Completed
TS-038-26	Scan barcode 5901234567890	Product found, added to cart
TS-038-27	Scan unknown barcode	"Product not found" message

ID	Scenario	Expected Result
TS-038-28	Sell product with stock = 0	Warning △ shown, sale proceeds, stock goes negative
TS-038-29	Void order within same shift (supervisor)	Order status → Voided, stock reversed, journal reversed
TS-038-30	Void order from different shift	Fail — "Orders can only be voided within the same shift"
Credit Sales		
TS-038-31	Credit sale to registered customer (within limit)	Payment method = Credit, customer balance updated
TS-038-32	Credit sale exceeding credit limit (online)	Fail — "Credit limit exceeded" with balance details
TS-038-33	Credit sale to Walk-In customer	Fail — "Walk-In customers cannot use credit"
TS-038-34	Credit sale offline within tolerance (10%)	Sale allowed, flagged as offline_credit
TS-038-35	Credit sale offline exceeds tolerance	Fail — "Offline credit limit exceeded"
TS-038-36	Sync reveals credit sale actually over-limit	Order flagged CREDIT_OVER_LIMIT, supervisor notified
Returns		
TS-038-37	Return 1 unit from 3-line order	Return order created, stock increased, journal reversed
TS-038-38	Return quantity > original	Fail — "Return quantity cannot exceed original"
TS-038-39	Cash refund on return	Shift total_cash_refunds updated, expected_cash reduced
TS-038-40	Return of credit sale	Customer balance reduced
Inventory Integration		

ID	Scenario	Expected Result
TS-038-41	Complete sale order → stock deduction	StockMovement (POS_SALE, OUT) created for store warehouse
TS-038-42	Complete return order → stock return	StockMovement (POS_RETURN, IN) created
TS-038-43	Void order → stock reversal	StockMovement (POS_VOID, IN) reverses original deduction
Finance Integration		
TS-038-44	Complete cash sale → journal entry	DR Cash, CR Revenue + Tax Payable
TS-038-45	Complete card sale → journal entry	DR Card Receivable, CR Revenue + Tax Payable
TS-038-46	Complete credit sale → journal entry	DR Accounts Receivable (sub: customer), CR Revenue + Tax Payable
TS-038-47	Cash refund → reversal journal	DR Revenue + Tax Payable, CR Cash
Offline & Sync		
TS-038-48	Create order while offline	Order saved in IndexedDB, syncStatus = 'pending'
TS-038-49	Reconnect after 3 offline orders	All 3 orders synced in batch, server assigns order numbers
TS-038-50	Duplicate sync submission (same local_id)	Server returns existing order, no duplicate created
TS-038-51	Product price changed server-side, terminal online	SignalR pushes PriceChanged event, product tile updates
TS-038-52	Full sync on first terminal activation	All products, customers, config downloaded to IndexedDB
TS-038-53	Delta sync with since parameter	Only records with modified_at > since returned

ID	Scenario	Expected Result
Module Integration		
TS-038-54	Enable POS module	POS menu appears in sidebar, schema objects accessible
TS-038-55	Disable POS module with open shifts	Fail — "Close all shifts before disabling POS" (grace period applies)
TS-038-56	Product form: POS tab visible when POS enabled + is_sellable	POS settings tab visible
TS-038-57	Product form: POS tab hidden when POS disabled	POS tab not rendered
TS-038-58	POS API call when POS module disabled	403 MODULE_NOT_LICENSED
Edge Cases		
TS-038-59	Offline for 48 hours, 200 orders queued	All orders sync successfully on reconnect. Order numbers assigned sequentially
TS-038-60	Two terminals sell last unit of same product simultaneously	Both sales succeed. Stock goes to -1. Negative stock notification created

20. Development Phases

20.1 Phase Overview

Phase	Focus	Days	Tasks
Phase 1	POS Schema, Stores, Terminals	4	M038.1-M038.3, M038.16 (partial), M038.14
Phase 2	Shifts, Cash Management, Cashier Sessions	5	M038.4-M038.7, shift lifecycle services
Phase 3	POS Orders, Order Lines, Payments	6	M038.8-M038.10, order processing service, payment service
Phase 4	Product/Customer POS Extensions, Sync Engine	5	M038.11-M038.12, sync API extensions, Dexie schema
Phase 5	Inventory + Finance Integration	4	M038.13, M038.18, M038.20, stock/journal services
Phase 6	SignalR Hub + PWA Client	7	PosHub, Service Worker, React PWA components, offline mode
Phase 7	Permissions, Back-Office UI, Testing	5	M038.15, M038.17, M038.19, admin screens, E2E tests
Total		36 days	20 migrations, 14 new tables, 3 modified tables

20.2 Phase Details

Phase 1 — POS Schema, Stores, Terminals (4 days)

Track	Tasks	Days
1A	M038.1 — Create <code>pos</code> schema + M038.14 — Activate POS module in catalog	0.5
1B	M038.2 — <code>pos.Stores</code> table + EF Core entity + StoreService	1.5
1C	M038.3 — <code>pos.Terminals</code> table + device binding flow	1
1D	Store + Terminal API controllers + validation	1

Phase 2 — Shifts, Cash Management, Cashier Sessions (5 days)

Track	Tasks	Days
2A	M038.4 — <code>pos.Shifts</code> table + shift state machine service	1.5
2B	M038.5 — <code>pos.CashMovements</code> table + cash movement service	1
2C	M038.6 — Add POS columns to <code>auth.Users</code> + PIN hashing service	1
2D	M038.7 — <code>pos.CashierSessions</code> + session switch logic	1.5

Phase 3 — POS Orders, Order Lines, Payments (6 days)

Track	Tasks	Days
3A	M038.8 — <code>pos.PosOrders</code> table + order number generation	1.5
3B	M038.9 — <code>pos.PosOrderLines</code> + line total calculation	1
3C	M038.10 — <code>pos.PosPayments</code> + split payment logic	1.5
3D	PosOrderService — complete order flow (discount, tax, payment validation)	2

Phase 4 — Product/Customer POS Extensions, Sync Engine (5 days)

Track	Tasks	Days
4A	M038.11 — <code>pos.ProductPosSettings</code> extension + Product API extension	1.5
4B	M038.12 — <code>pos.CustomerPosSettings</code> + Customer API extension	1
4C	Extend <code>/api/sync/catalog</code> and <code>/api/sync/customers</code> with POS settings	1
4D	New sync endpoints: <code>/api/sync/pos-config</code> , <code>/api/sync/pos-users</code>	1.5

Phase 5 — Inventory + Finance Integration (4 days)

Track	Tasks	Days
5A	M038.13 — StockMovementType enum extension + PosStockService	1.5
5B	PosJournalService — journal entry creation per payment method	1.5
5C	M038.18 + M038.20 — POS routes + workflow rules	1

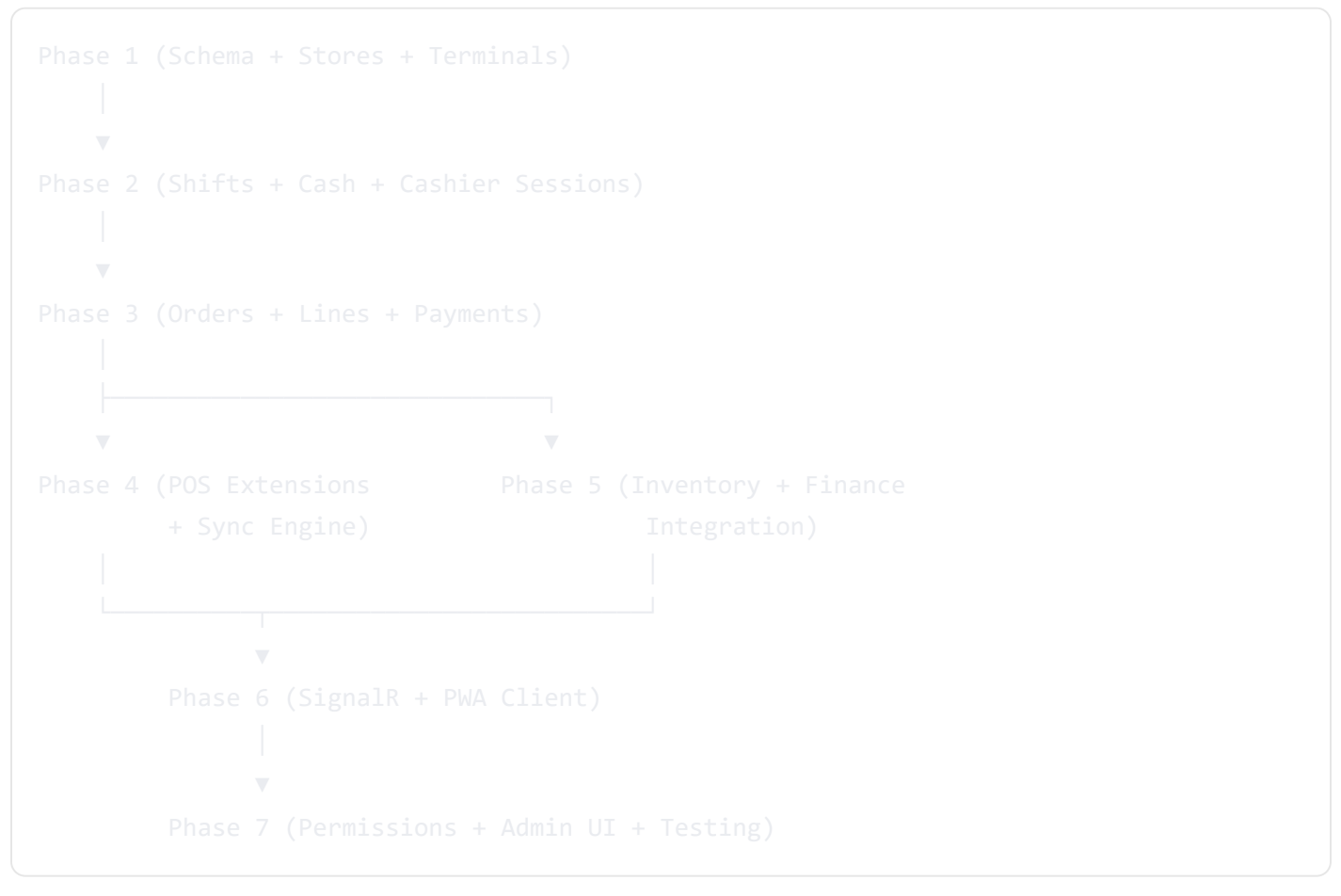
Phase 6 — SignalR Hub + PWA Client (7 days)

Track	Tasks	Days
6A	PosHub (SignalR) — hub setup, group management, events	1
6B	Dexie.js schema + SyncEngine class	1.5
6C	Service Worker (Workbox) — shell caching, offline fallback	1
6D	React PWA — selling screen, product grid, cart, search/barcode	2
6E	React PWA — payment screen, receipt, shift management	1.5

Phase 7 — Permissions, Back-Office UI, Testing (5 days)

Track	Tasks	Days
7A	M038.15 + M038.17 — POS features + permissions seed	0.5
7B	M038.19 — POS indexes optimization	0.5
7C	Back-office: Store list/detail, terminal management, POS product tab	1.5
7D	Back-office: Shift history, order history, POS reports	1
7E	E2E testing — full sale flow, offline flow, sync flow, shift close flow	1.5

20.3 Phase Dependencies



21. Future Enhancements

The following are **not included** in this addendum and will be addressed in future addendums:

#	Feature	Target	Description
1	Integrated Payment Processing	Addendum 39+	Stripe Terminal, Adyen, Square integration via <code>provider_type</code> / <code>provider_config</code>
2	Restaurant / F&B Module	Future	Table management, kitchen display system (KDS), course sequencing
3	Franchise Model	Future	Inter-company POS, franchisee P&L, royalty calculation
4	Loyalty Program	Future	Points earn/redeem at POS, tier management
5	Advanced Promotions	Future	BOGO, bundle deals, time-based promotions, coupon codes
6	Receipt Printing	Future	ESC/POS thermal printer support, cash drawer kick command
7	POS Analytics Dashboard	Future	Real-time sales dashboard, top products, hourly breakdown
8	Multi-Currency POS	Future	Currency selection per transaction, forex rates
9	E-Commerce Integration	Future	Unified stock + orders across POS and online channels
10	Customer Display	Future	Secondary screen showing items as scanned
11	Inventory Count from POS	Future	Cycle count workflows initiated from POS terminal
12	Product Variants at POS	Future	Size/color selection on POS product tile (depends on PRODUCT_VARIANTS feature)
13	Offline Shift Open	Future	Allow supervisor to open shift without server connectivity
14	Scale Integration	Future	Weighing scale USB/Bluetooth integration for weighable items

Appendix A — Entity Relationship Summary

pos.Stores (NEW)

- └─ M:1 → inventory.Warehouses (1:1 binding)
- └─ M:1 → lookups.TaxRates (default tax)
- └─ 1:M → pos.Terminals (NEW)
 - └─ 1:M → pos.Shifts (NEW)
 - └─ 1:M → pos.CashierSessions (NEW)
 - └─ 1:M → pos.PosOrders (NEW)
 - └─ 1:M → pos.PosOrderLines (NEW)
 - └─ M:1 → inventory.Products
 - └─ 1:M → pos.PosPayments (NEW)
 - └─ M:1 → customers.Customers
 - └─ self-ref → pos.PosOrders (returns)
 - └─ 1:M → pos.CashMovements (NEW)
 - └─ M:1 → auth.Users (device_uuid bind)
 - └─ M:M → auth.Users (store staff via default_store_id)

pos.ProductPosSettings (NEW) — 1:1 → inventory.Products

pos.CustomerPosSettings (NEW) — 1:1 → customers.Customers

auth.Users (MODIFIED — pos_pin, pos_role, default_store_id added)

Appendix B — Offline Capability Matrix

Feature	Online	Offline	Notes
Browse products	✓ Real-time	✓ Cached	Delta sync every 5 min
Barcode scan	✓	✓	Local barcode lookup
Search products	✓	✓	Local Dexie.js search
Search customers	✓	✓	Local Dexie.js search
Create order	✓	✓	Stored in IndexedDB
Split payment	✓	✓	Recorded locally
Cash payment	✓	✓	Recorded locally
Card payment	✓	✓	Recorded locally (manual entry)
Credit sale	✓ Real-time check	⚠ Cached limit + tolerance	Flagged on sync if over-limit
Apply discount	✓	✓	Local calculation
View receipt	✓	✓	Generated locally
Email receipt	✓	✗	Queued for sync
Cash in/out	✓	✓	Recorded locally
Open shift	✓	✗	Requires server auth
Close shift (count)	✓	⚠	Count entered, final close on sync
Return order	✓	⚠	Only if original in local DB
Void order	✓	✗	Requires server confirmation
Create customer	✓	⚠	Local temp record, synced later
PIN login	✓ Server verify	✓ Cached hash	Bcrypt comparison
SignalR updates	✓	✗	Reconnect triggers delta sync

Appendix C — POS Module Feature Flags

Feature Code	Name	Type	Available	Description
POS_ORDERS	POS Order Processing	Core	✓	Create and manage POS sale orders
POS_SHIFTS	Shift Management	Core	✓	Open/close/reconcile shifts with cash tracking
POS_CASHIER_PIN	Cashier PIN Authentication	Core	✓	Shared terminal PIN-based cashier switching
POS_PAYMENTS	Payment Recording	Core	✓	Record cash, card, mobile payment methods
POS_OFFLINE	Offline Mode	Core	✓	Process transactions without internet
POS_RETURNS	POS Returns & Refunds	Toggleable	✓	Process returns and refunds at POS
POS_CREDIT_SALES	Credit Sales (On-Account)	Toggleable	✓	Allow credit sales to registered customers
POS_BARCODE	Barcode Scanning	Toggleable	✓	Camera or scanner based barcode lookup
POS_MULTI_STORE	Multi-Store Management	Toggleable	✓	Manage multiple retail locations
POS_ANALYTICS	POS Analytics Dashboard	Toggleable	✗ (Future)	Sales analytics and reporting
POS_PROMOTIONS	Promotions Engine	Toggleable	✗ (Future)	Advanced promotions and coupon management
POS_LOYALTY	Loyalty Integration	Toggleable	✗ (Future)	Customer loyalty points at POS

End of FSD Addendum 38